


Ministerul Educației și Cercetării
Universitatea OVIDIUS din Constanța
Facultatea de Matematică și Informatică
Specializarea Computer Science

# Information System for Network Detection Intrusion

Bachelor Thesis

**Scientific Advisor:**
**Lect. univ. dr. Dorin Iordache**

**Graduate:**
**Bora Cristian-Vasile**

Constanța
Iulie 2026

# Acknowledgements

A few people deserve more than the line I can give them here.

First, my coordinator, Prof. Dorin Iordache. He let me pick a topic that was bigger than what I could probably finish, then steered me through the parts I would have got wrong on my own — the threat model, the way to frame the hybrid detection, the decision to keep the simulator instead of chasing a live red-team scenario that would have cost me weeks. The conversations after the project review, when I was second-guessing the whole thing, were the ones that kept me on track. Thank you.

To my colleagues from the Computer Science cohort — the ones who let me ramble about Scapy at coffee, who pointed out that my JWT expiry was off by a factor of sixty (it was supposed to be minutes, not seconds, and that's an embarrassing kind of bug), and who told me when the UI looked like a 2010 admin panel before I'd settled on the Deep Navy theme. You know who you are.

To my family, who put up with the late nights and the muttering about Npcap on Windows. None of you had any idea what I was doing on the laptop, and you let me do it anyway. That counts for a lot.

And to whoever decided that Romanian student WiFi should run on a single overworked router — thank you for forcing me to write a deterministic simulator instead of relying on live network capture. The thesis is better for it.

*Bora Cristian-Vasile*
*Constanța, 2026*

# Contents

**Acknowledgements** **ii**

**1 Introduction** **1**

1.1 Abstract . . . . . 1

1.2 Motivation . . . . . 2

1.3 Problem Statement . . . . . 4

1.4 Contributions . . . . . 4

1.5 Thesis Structure . . . . . 5

**2 Theoretical Background** **6**

2.1 Network Intrusion Detection Systems . . . . . 6

2.2 Signature-Based Detection . . . . . 7

2.3 Anomaly-Based and Machine-Learning Detection . . . . . 8

2.3.1 Random Forest . . . . . 8

2.3.2 Isolation Forest . . . . . 9

2.4 Hybrid Detection . . . . . 9

2.5 Forensics . . . . . 10

**3 Technologies and Theoretical Elements Used** **12**

3.1 Python . . . . . 12

3.2 FastAPI . . . . . 13
3.3 SQLAlchemy 2 (Asynchronous) . . . . . 14
3.4 aiosqlite and SQLite . . . . . 14
3.5 Pydantic v2 . . . . . 15
3.6 scikit-learn . . . . . 16
3.7 Scapy . . . . . 16
3.8 NumPy . . . . . 17
3.9 React 18 . . . . . 17
3.10 Vite . . . . . 18
3.11 TailwindCSS . . . . . 18
3.12 Recharts . . . . . 18
3.13 lucide-react . . . . . 19
3.14 date-fns . . . . . 19
3.15 WebSocket Protocol . . . . . 19
3.16 JWT and bcrypt . . . . . 20
3.17 Joblib . . . . . 20
3.18 PyYAML . . . . . 20
3.19 Uvicorn . . . . . 20
3.20 Node.js and npm . . . . . 21
3.21 Operating System Considerations . . . . . 21

**4 Related Work** **22**

4.1 Snort . . . . . 22
4.2 Suricata . . . . . 23
4.3 Zeek . . . . . 24
4.4 Wazuh . . . . . 24
4.5 Splunk Enterprise Security . . . . . 25
4.6 Academic Machine-Learning IDS . . . . . 26

4.7 Positioning of This Work . . . . . 26

**5 Design and Implementation** **28**

5.1 General Architecture . . . . . 28

5.1.1 The Two-Tier Split . . . . . 28

5.1.2 Packet Life Cycle . . . . . 29

5.2 Capture Layer . . . . . 30

5.2.1 LiveSniffer (Scapy) . . . . . 30

5.2.2 TrafficSimulator . . . . . 31

5.2.3 Mode Selection . . . . . 32

5.3 The Dispatcher . . . . . 32

5.4 Signature Engine . . . . . 33

5.4.1 Match Specification . . . . . 33

5.4.2 Threshold Tracking . . . . . 34

5.4.3 Evaluate Loop . . . . . 34

5.5 Machine-Learning Detector . . . . . 35

5.5.1 Feature Vector . . . . . 35

5.5.2 Training . . . . . 36

5.5.3 Inference . . . . . 37

5.6 Hybrid Merge Logic . . . . . 37

5.7 Forensics Module . . . . . 38

5.7.1 Downloading from the UI . . . . . 38

5.8 REST and WebSocket API . . . . . 39

5.9 Frontend . . . . . 40

5.9.1 Authentication and Routing . . . . . 41

5.9.2 Live Data Lifting . . . . . 41

5.9.3 Pages . . . . . 41

5.10 Database Schema . . . . . 44

5.11 Authentication Subsystem . . . . . 46
5.12 The Attack Console (Red Team in a Box) . . . . . 46
5.13 The Cross-Platform Launcher . . . . . 48

**6 Experimental Validation** **49**
6.1 Test Environment . . . . . 49
6.2 Synthetic Validation . . . . . 50
6.2.1 Setup . . . . . 50
6.2.2 Results . . . . . 50
6.2.3 Hybrid Detection . . . . . 51
6.3 Performance Measurements . . . . . 52
6.3.1 Per-Packet Cost . . . . . 52
6.3.2 End-to-End Latency . . . . . 52
6.3.3 Memory Footprint . . . . . 53
6.3.4 Latency of Hybrid Categorisation . . . . . 53
6.4 Discussion . . . . . 53

**7 Conclusions and Future Work** **55**
7.1 Achievements . . . . . 55
7.2 Limitations . . . . . 56
7.3 Future Work . . . . . 57
7.4 Closing Remarks . . . . . 58

**Bibliography** **60**

# List of Figures

5.1 High-level architecture: a packet enters at the source (a NIC via Scapy, or the in-process simulator), is captured, handed to the central dispatcher, evaluated by both the signature and ML engines, and exits as a database row, a JSONL file on disk and a WebSocket message to the browser. . . 30

5.2 The ML Models page after training the default Random Forest classifier. The four metric cards report accuracy, precision, recall and F1 on the held-out test split; the bar chart shows the per-feature importance derived from the model's `feature_importances_` attribute, with `src_port` and `length` carrying the most discriminative signal. . . . . 36

5.3 The Forensics page listing the JSONL captures stored on disk. Each row corresponds to one alert and carries a link to the underlying `Alert` record plus a Download button that uses the authenticated `apiDownload` helper. In this run the dispatcher has accumulated several hundred captures, each holding 50 packets at roughly ten kilobytes per file. . . . . 39

5.4 The Login page. A single form with the OAuth2 password flow against `POST /api/auth/login`; on success the token is written to `sessionStorage` and the user is redirected to the Dashboard. . . . . 42

5.5 The Operations Dashboard. The visible spike on the live packet-rate chart is a single attack burst injected through the Attack Console; the 26 alerts produced by that burst show up across the severity pie, the 6-hour timeline (the tall bar around 01:30), the detection-method pie and the live alert feed on the right. . . . . 42

5.6 The Alerts page after a port-scan burst. Every row's detection method is `signature` and the source IP matches the synthetic attacker pool (`185.220.101.5`); the destination IP `10.0.0.10` is the configured defended host. The two `Telnet cleartext login` entries are a different rule that happened to fire on traffic to port 23 from the same attacker burst. . . . . 43

5.7 The Alert Detail page for a single Nmap TCP SYN scan alert. The header summarises the detection (severity, method, category, timestamp); the metadata block lists the 5-tuple and the path of the JSONL capture; and the packet timeline at the bottom shows the last 31 packets captured around the alert, all TCP SYN frames from the attacker hitting different destination ports on the defended host — the textbook signature of a port scan. . . . . 44

5.8 The Rules page listing the ten signature rules seeded from `default_rules.yaml`. The `Match` column shows the raw JSON the engine evaluates; the Severity and Category columns are visual cues for triage; Edit and Delete buttons on each row mutate the database and reload the in-memory rule list without a backend restart. . . . . 45

5.9 The Settings page. The Account panel on the left lets the signed-in user rotate their own password (the form posts to `POST /api/auth/change-password`, which re-verifies the current password before writing a new bcrypt hash). The Engine Status panel on the right polls `/api/health` every three seconds; the Deployment Configuration block at the bottom is a reference for which `.env` variables control which behaviour. . . . . 45

5.10 The Attack Console at rest. The Defended Host panel at the top reports which IP every simulated attack will target; the "How this works" banner explains the real-time inspection model; the Benign traffic baseline toggle is off by default so the dispatcher sees no packets at all until an attack button is clicked. The six attack cards expose the catalogue from `TrafficSimulator.ATTACK_CATALOG`. . . . . 47

5.11 The Attack Console immediately after the operator has fired the Nmap port scan. The Recent Injections row at the bottom shows the attacker IP from the simulator's pool (`185.220.101.5`), the defended host as target (`10.0.0.10`), 39 packets injected, and the green *39 fired* chip confirming the dispatcher produced 39 alerts from the burst. This per-attack feedback loop replaces the older "switch to the Alerts tab to check" workflow. . . . . 47

# List of Tables

4.1 Feature comparison of Ovidius NIDS against related tools. "✓" indicates the feature is present out of the box, "∼" indicates partial or plugin-based support, "–" indicates the feature is outside the tool's scope, and "×" indicates the feature is absent. . . . . 27

6.1 Synthetic attack validation: packets injected and alerts generated, averaged over five runs per attack kind. . . . . 51

6.2 Detection method breakdown with Random Forest activated. . . . . 51

6.3 Per-packet processing cost (median of 5000 samples). . . . . 52

# Chapter 1

# Introduction

## 1.1 Abstract


### Rezumat (Limba Română)

Ovidius NIDS este un sistem de detecție a intruziunilor în rețea (NIDS), construit în jurul a trei subsisteme cooperante: un motor de reguli scris manual care compară pachetele cu o bază de date de semnături inițializată din YAML, un clasificator scikit-learn care scorează aceleași pachete folosind fie Random Forest (supervizat), fie Isolation Forest (nesupervizat), și un înregistrator de forensică ce păstrează un mic inel de pachete în jurul fiecărei alerte. Cele trei subsisteme partajează același event loop, aceeași reprezentare a pachetelor și același strat de stocare, astfel încât orice alertă conține contextul complet care a produs-o.

Codul include un backend FastAPI cu API-uri REST și WebSocket, un strat asincron SQLAlchemy peste SQLite, un frontend React (Vite + TailwindCSS) și un launcher one-click pentru Windows, macOS și Linux. O *Consolă de Atac* integrată injectează șase clase de atac (scan de porturi, brute-force SSH, flood ICMP, injecție SQL, injecție de comenzi, reverse shell) prin același dispatcher pe care l-ar alimenta o interfață de rețea reală. Latența end-to-end, de la sosirea pachetului până la notificarea în UI, a rămas sub 250 ms pe un laptop de gamă medie.

## Abstract (English)

Ovidius NIDS is a Network Intrusion Detection System built around three cooperating subsystems: a hand-written rule engine that matches packets against a YAML-seeded signature database, a scikit-learn classifier that scores those same packets with either Random Forest (supervised) or Isolation Forest (unsupervised), and a forensic recorder that persists a small ring of packets around every alert. The three share one event loop, one packet representation, and one storage tier, so any alert carries the full context that produced it.

The codebase covers a FastAPI backend with REST and WebSocket APIs, an asynchronous SQLAlchemy layer on top of SQLite, a React frontend (Vite + TailwindCSS), and a one-click cross-platform launcher. An integrated *Attack Console* injects six attack classes (port scan, SSH brute-force, ICMP flood, SQL injection, command injection, reverse shell) through the same dispatcher path a live network interface would feed. End-to-end alert latency, from packet arrival to UI notification, stayed under 250 ms on a mid-range laptop.



## Cuvinte cheie / Keywords

**Domeniu principal / Main domain:** Securitatea rețelelor, Sisteme de detecție a intruziunilor (NIDS) / Network Security, Network Intrusion Detection.

**Domenii adiacente / Adjacent domains:** Învățare automată, forensică digitală, dezvoltare full-stack web, inginerie software / Machine Learning, Digital Forensics, Full-Stack Web Development, Software Engineering.

**Tipul lucrării / Work type:** Cercetare și sinteză (research and synthesis): the thesis combines a survey of the NIDS literature with an original engineering implementation evaluated through a reproducible synthetic-traffic methodology.


# 1.2 Motivation

There's a gap between what intrusion-detection researchers publish and what the people guarding actual networks deploy, and it's been bothering me since I started reading the papers. Pick up the proceedings of any recent IEEE Security and Privacy conference. You'll find dozens of papers throwing machine-learning models at the KDD'99 or CICIDS2017 datasets and reporting F1 scores so high you'd think the problem was

solved [4, 12]. Then look at what a real Security Operations Centre runs in production. Snort signatures. Or Suricata signatures. Fed into a Splunk dashboard. That's it. The literature talks about models. The operators want workflows [26].

Why the disconnect? A signature engine emits a clear alert with a rule name on it. That's what an analyst needs at three in the morning when a Slack notification wakes them up: a name, a verdict, something to act on. An ML classifier hands them a probability score. Probabilities are harder to triage, easier to tune wrong, and they generate enough false alarms that operator trust eventually erodes. So when the two paradigms are framed as alternatives, signatures win on the operational floor every time — whether or not they win on paper.

I didn't want to pick a side. The premise of this project is that signatures and ML don't have to compete. They can cooperate. Think of it as three categories of alert. The first: a packet matches a high-confidence rule, end of story. The second: no rule matched, but the classifier scored the packet high enough to be suspicious — the case where a second opinion is useful. The third: both engines flagged the same packet. That last one is the strongest signal you'll get. By labelling every alert with the engine that produced it, the console tells the analyst not just *what* fired but *why* they should care.

There was a second reason I started, and it came from looking at the existing tools. Snort, Suricata and Zeek are good engines. Really good. But they're all command-line daemons. There's no analyst console in any of them. The alert page, the packet pivot, the rule editor — everything has to be bolted on by the operator. On the other side of the market sit Splunk Enterprise Security and IBM QRadar. Beautiful UIs. But the licences run into five figures a year and the products don't even do detection themselves — they aggregate alerts from external engines. So there's a gap in the middle: a single self-contained app, self-hostable, that does detection, alerting and forensics in one binary. Ovidius NIDS is my attempt at occupying that gap.

A third reason was more pedagogical. A thesis is also a chance to show you can ship working software. I wanted a codebase where somebody could double-click an icon, log in with default credentials, and see actual alerts in their browser inside thirty seconds — no wiki, no package manager, no infrastructure. That target shaped several decisions. SQLite instead of Postgres. A virtualenv instead of Docker. A built-in traffic simulator so the dashboard isn't empty on first boot. The result is a system that runs end-to-end on a laptop with nothing else installed. That makes it useful for academic demonstration, and it works fine as a starting point for deployment on a small network too.

## 1.3 Problem Statement

This thesis tries to answer three concrete engineering questions.

First: **can a single application combine signature-based and machine-learning detection in a way that makes the cooperation visible?** Most existing systems either run one or the other. A few run both, but in parallel silos with no shared output schema. I wanted alerts to carry an explicit *detection method* label — "signature", "ml" or "hybrid" — and I wanted the hybrid label to be earned, not assigned. Both engines have to flag the same packet for it.

Second: **can the system keep enough context per alert that an analyst can investigate without reaching for an external packet sniffer?** The canonical workflow when an alert fires is "what packets led to this?". In a normal NIDS deployment the operator opens a separate full-packet-capture tool — tcpdump, Moloch (now Arkime), Wireshark — and correlates by timestamp and 5-tuple. That's two consoles, two indexing layers, and a lot of chances to get the correlation wrong. I wanted that whole step removed. Every alert should show up with the last fifty packets attached, downloadable from the same UI that displayed the alert.

Third: **can the whole thing be deployed without infrastructure expertise?** A familiar failure mode of academic prototypes is that they need a graduate-level setup procedure before they'll even start. I set myself a hard requirement: someone who isn't a developer should be able to launch the entire stack with a single double-click on Windows, macOS or Linux, without installing dependencies by hand.

## 1.4 Contributions

The contributions of this thesis are:

1. A modular detection pipeline (see Chapter 5, Section 5.6) in which signature evaluation and ML scoring run on the same packet stream and produce three explicit alert categories. The merge logic is contained in a single method, `Dispatcher._evaluate`, roughly twenty lines long.

2. A YAML-seeded rule store that is also editable live through the UI without restarting the engine. The rule compilation step (Section 5.4) builds an in-memory list of compiled match predicates which is hot-swapped whenever the database changes.

3. A per-alert JSONL ring-tail capture and a UI page that lets the analyst replay the packets and download the file via an authenticated endpoint (Section 5.7). The capture is line- delimited rather than libpcap-formatted so the analyst can read it in any text editor.

4. A React analyst console (Section 5.9) with eight pages: Login, Dashboard, Alerts, Alert Detail, Rules, ML Models, Forensics, Attack Console and Settings. Live updates use two WebSocket channels (`/ws/alerts` and `/ws/traffic`) which survive page navigation by being scoped to the shared layout component.

5. An integrated *Attack Console* (Section 5.12) that exposes six attack types (Nmap scan, SSH brute-force, ICMP flood, SQL injection, command injection, reverse shell) as buttons in the UI. Each invocation injects a small burst of synthetic packets through the dispatcher exactly as a live capture would, returns the number of alerts generated and gives instant audience feedback during demonstration.

6. A one-click cross-platform launcher (Section 5.13) which creates a Python virtual environment, installs both the Python and Node dependencies, starts FastAPI and Vite as supervised subprocesses, waits for both to be reachable, opens the browser and handles `Ctrl+C` with an idempotent shutdown routine.

7. A validation methodology (Chapter 6) that drives the entire attack catalogue through the dispatcher and measures the alerts generated per attack kind, the latency from injection to notification, and the agreement between the signature and machine-learning engines.

## 1.5 Thesis Structure

The rest of the thesis is organised like this.

Chapter 2 covers the theory I needed: NIDS as a discipline, signature versus anomaly paradigms, the two ML algorithms I picked, and the forensic idea that ties everything together. Chapter 3 walks through the specific technologies in the stack and why each one ended up there. Chapter 4 positions the project against the production-grade tools (Snort, Suricata, Zeek, Wazuh, Splunk ES) and the published ML-IDS research. Chapter 5 is the big technical chapter; it works through every component of the implementation with file references and code listings. Chapter 6 describes the validation methodology and reports the latency and accuracy numbers I collected. Chapter 7 wraps up with what got done and what's left for future work.

# Chapter 2

# Theoretical Background

## 2.1 Network Intrusion Detection Systems

A Network Intrusion Detection System inspects packets crossing a network and emits an alert when those packets match a model of malicious activity. The discipline grew out of host-based intrusion detection in the late 1980s and was formalised in the late 1990s when Vern Paxson published Bro — the system later renamed Zeek [17] — and Martin Roesch published the first version of Snort [23]. Both systems are still maintained today and still influence the architecture of every serious open-source NIDS, including this one.

A NIDS sits passively in front of the traffic it watches. It does not forward, drop or modify packets; it only observes. Inline detection, where the system actively blocks traffic, falls under the related acronym NIPS (Network Intrusion Prevention System). Ovidius NIDS is a NIDS in the strict sense: it can produce alerts but it cannot stop a packet from reaching its destination. The reasoning is partly philosophical and partly practical. Blocking traffic at the inspection point creates a single point of failure for the whole network, and production deployments overwhelmingly prefer to run detection in a side-channel (a SPAN port, a TAP, a virtual interface) and to put prevention behind a separate firewall layer.

The taxonomy of detection strategies usually splits along two axes: the *paradigm* (misuse-based versus anomaly-based) and the *granularity* (packet-level versus flow-

level) [8]. The paradigm axis is the more important one. Misuse-based or signature-based detection encodes the patterns of known attacks; the engine matches each packet against a curated set of rules. Anomaly-based detection encodes a model of normal traffic; the engine flags packets that deviate from that model. The granularity axis is more of an implementation choice: a packet-level engine inspects each frame in isolation, while a flow-level engine aggregates packets sharing the same 5-tuple into a bidirectional flow before scoring. Ovidius NIDS operates packet-level because the design priority was real-time responsiveness; flow features such as inter-arrival time are kept as streaming state inside the ML detector.

## 2.2 Signature-Based Detection

Signature-based detection is the older and conceptually simpler approach. The engine holds a database of patterns; for each packet it iterates the patterns and emits an alert whenever one matches. The patterns themselves are typically expressed in a domain-specific language. Snort's rule syntax has become an informal standard and is shared (with extensions) by Suricata [16]. A typical Snort rule looks like this:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 22
  (msg:"SSH brute force attempt";
   flags:S;
   threshold:type both, track by_src, count 10, seconds 30;
   classtype:attempted-recon; sid:1000001; rev:1;)
```

The header (`alert tcp ... -> ... 22`) constrains the protocol, direction and ports. The options inside the parentheses constrain payload contents, TCP flag combinations, packet sizes, and so on. The `threshold` option turns a single-packet pattern into a temporal one — it fires only when a certain count of matches arrives within a given window from the same source.

Signature-based detection has two well-understood strengths. It is *precise* (false-positive rate is low when rules are well-written) and it is *auditable* (each alert carries the identifier of the rule that produced it, so an analyst can trace the decision back to a human-written predicate). Its corresponding weakness is that it cannot detect attacks for which no rule has yet been written. Every novel attack technique has a window between its first appearance in the wild and the day a signature for it ships in the rule distribution [8]; during that window, a signature-only engine sees nothing.

Ovidius NIDS uses a JSON-based match specification instead of the Snort DSL. The decision was driven by two factors. First, a JSON match maps cleanly to a Pydantic model on the backend and to a JavaScript object on the frontend, which makes the rule-editing experience much simpler than parsing and pretty-printing Snort syntax. Second, the project does not aim to compete with Snort on rule expressiveness; it aims to cover the categories of attack the validation scenario uses (port scans, brute-force attempts, simple floods, content-based exploits), and the JSON specification is sufficient for all of them. Section 5.4 describes the exact schema.

## 2.3 Anomaly-Based and Machine-Learning Detection

Anomaly-based detection inverts the signature paradigm. Instead of asking "does this packet look like a known attack?" it asks "does this packet look like normal traffic?". The engine learns a model of the normal distribution from observed data and flags samples that fall outside it. The literature on the topic is enormous; Buczak and Guven counted over a hundred distinct proposals in a single survey paper [4], and Khraisat et al. extended the count in a more recent review [12].

I selected two algorithms from this body of work for Ovidius NIDS: Random Forest and Isolation Forest. Both are tree-based ensemble methods from scikit-learn [18], and both are fast enough to run in the per-packet inference path without affecting the overall throughput. They were chosen for their complementary semantics rather than for any claim of state-of-the-art accuracy.

### 2.3.1 Random Forest

The first algorithm came from Leo Breiman's 2001 paper [3]. The construction is straightforward to describe but surprisingly hard to beat in practice. You grow a collection of decision trees, each on its own bootstrap sample of the training rows, and at every split inside each tree you let the algorithm pick from a random subset of the features rather than from all of them. At prediction time the trees vote. The randomness in the two sampling steps is what stops the forest from agreeing too closely with itself — the resulting ensemble keeps the interpretability of individual trees but throws away most of their variance.

I picked it for the NIDS pipeline because of three properties that mattered to me. The first is calibrated probabilities, which gave me a sensible threshold knob (`0.8` by default) between flagging too much and flagging too little. The second is robustness: the

algorithm tolerates noisy or partially irrelevant features without obvious degradation, and I knew upfront that not all six of my features (Section 5.5.1) would be equally informative on every packet. The third is the `feature_importances_` attribute on the trained model. Surfacing it as a bar chart on the ML Models page (Figure 5.2) means an analyst can see, at a glance, that the classifier weighted `inter_arrival_ms` heavily for a particular alert — that's actionable; a bare probability score isn't.

The downside is that it needs labelled training data. For this thesis the training set is synthesised from known patterns and that works fine. In production deployment, keeping labels fresh enough to retrain on becomes the dominant ongoing cost, which is one of the reasons I wanted the second algorithm below as a backup.

### 2.3.2 Isolation Forest

The second algorithm came from a 2008 paper by Liu, Ting and Zhou [14], and the intuition behind it is charming once you see it. Anomalies, the authors argue, are by definition both rare and different, so a random partitioning scheme should be able to isolate them faster than it isolates the rest of the data. The construction reflects that idea directly. Grow a forest of binary trees, each built by picking a random feature and a random split point between that feature's minimum and maximum value. To score a new sample, drop it through every tree and measure how deep it had to travel before reaching a leaf. Anomalies travel less. The shorter the average path, the more anomalous the sample is considered to be.

The advantage over a Random Forest is one I keep coming back to: no labels. You only need a clean sample of normal traffic, which is much easier to collect than a labelled attack corpus. The disadvantage shows up at the edges of the normal distribution, where rare-but-legitimate behaviour starts looking suspicious in exactly the way attacks do. That's the exact failure mode that Sommer and Paxson called out [26] in their critique of academic ML-IDS, and it's the reason I shipped *both* algorithms behind the same interface rather than committing to one. An operator who has labelled data gets the supervised Random Forest. One who doesn't gets the unsupervised Isolation Forest. The rest of the pipeline doesn't change.

## 2.4 Hybrid Detection

When both engines run on the same packet stream, the dispatcher can produce three different alert categories:

**Signature-only:** the rule engine matched but the ML model classified the packet as benign. This is the highest-precision category — the rule guarantees the packet matches a known pattern, and the model's disagreement carries no penalty.

**ML-only:** no rule matched but the model's score exceeded the alert threshold. This category catches patterns for which no signature exists, at the cost of less explanation. An analyst looking at an ml-only alert knows the model thought the packet was malicious, but has to dig into the feature vector to see why.

**Hybrid:** both engines flagged the same packet. This is the most reliable category. The rule contributes precision (a human decided this pattern is malicious) and the model contributes corroboration (the statistical evidence supports the rule).

The categorisation is performed in the dispatcher's `_evaluate` method. Each candidate alert carries an explicit `detection_method` field that propagates all the way to the UI, where the analyst sees a coloured chip indicating the source. This removes the black-box critique that has historically been levelled at ML-based IDS [26]: the alert always tells the user which engine produced it, and a click on the alert detail page shows both the rule that matched and the ML score on the same screen.

## 2.5 Forensics

The forensic dimension is what distinguishes operational SIEM tools (Wazuh, Splunk, IBM QRadar) from detection-only engines. An alert by itself is just a row in a log; what makes it actionable is the context. The classic forensic workflow is: see the alert, retrieve the packets that produced it, eyeball them, decide if the alert is a true or false positive, take the appropriate downstream action.

In a typical NIDS deployment, the retrieval step is delegated to a full-packet-capture system that operates in parallel with the detection engine. Moloch / Arkime is the canonical open-source example. The architecture is sound but it involves running two separate stacks, correlating their data by timestamp and 5-tuple, and training the analyst on two different consoles. Ovidius NIDS takes a different position: every alert carries its own packet capture inline.

The mechanism is straightforward. The dispatcher maintains a fixed-size ring buffer (`deque(maxlen=200)`) of the most recent packets seen. When an alert is produced, the dispatcher slices the last 50 entries off the ring and serialises them to a JSON Lines file named after the alert ID. The path is stored on the `Alert` row and a separate

`PcapRecord` row indexes it for the UI. Download is served by an authenticated FastAPI endpoint that streams the file with a `FileResponse`. Section 5.7 walks through the implementation.

JSON Lines was chosen over the libpcap binary format for two practical reasons. JSONL is human-readable, so an analyst can open the file in any text editor or pipe it through `jq` for filtering. And JSONL does not depend on libpcap or wpcap being installed at the time the file is created, which simplifies the Windows deployment story considerably.

# Chapter 3

# Technologies and Theoretical Elements Used

This chapter describes the specific technologies that ended up in Ovidius NIDS's stack and the reasoning behind each choice. The list is long because the project spans backend, frontend, machine learning, packet capture and packaging. Each section is self-contained and may be read in any order.

## 3.1 Python

The whole backend runs on **Python** [20] 3.12. I picked the version deliberately: the launcher refuses anything newer than 3.13 because scikit-learn and numpy did not yet ship Windows wheels for 3.14 at the time I started this work, which would have forced a source build needing the Visual C++ build tools — a seven-gigabyte detour every evaluator of this thesis would have hit on a fresh Windows laptop. `run.py` checks the interpreter version on the first banner line and prints an upgrade path when the check fails; I added that gate the day I watched a friend spend forty minutes trying to compile numpy from source on 3.14.

Three reasons drove the language choice. The first is the scientific Python stack. `from sklearn.ensemble import RandomForestClassifier` on line one of a file is hard to beat when the alternative is linking against a C++ library and maintaining a binding

layer. The second is asyncio. Python's async primitives in 3.11+ are mature enough to host a real-time event loop without dropping into threads everywhere, and FastAPI is built directly on that substrate, so the entire packet pipeline can stay in one coroutine context. The third is pedagogical: Python is the language students at my faculty read fluently, and a defence committee evaluating the thesis should not have to learn a new syntax to follow the code.

The codebase uses type hints throughout (`from __future__ import annotations`) and validated by static review rather than a strict type checker. The annotations are not enforced at runtime by mypy because the project does not currently require that discipline, but the information is present for any future developer who wants to add it.

## 3.2 FastAPI

The REST and WebSocket surface of Ovidius NIDS is served by **FastAPI** [21], the asynchronous Python framework Sebastián Ramírez published in 2018. Under the hood FastAPI stacks on top of **Starlette** (the ASGI toolkit that handles raw HTTP and WebSocket plumbing) and **Pydantic** (the runtime validation library used for every request and response body in the project).

I evaluated Flask, Django and aiohttp before settling on FastAPI. What pushed me to commit was four properties, in roughly this order. The dependency-injection system came first. Every protected route in the codebase carries `Depends(get_current_user)` on its signature, and the framework handles JWT decoding, the database lookup and the 401 emission without a line of boilerplate inside the route body — one of the few cases in Python where a decorator genuinely earns its keep. Then the auto-generated OpenAPI documentation at `/docs`: zero configuration, every endpoint browsable and testable from the Swagger UI in the browser. I used this constantly during development to fire test requests at the alerts filter and the WebSocket auth without ever writing curl commands. Third, the Pydantic v2 integration; request and response bodies are declared as typed classes and the framework moves JSON across the wire as Python objects with validation on both sides. Fourth, the asynchronous-by-default story — every endpoint in Ovidius NIDS is an `async def` function and the WebSocket handlers share the same event loop as the dispatcher, which mattered more than I expected once the live packet stream started filling up the loop.

FastAPI runs under **Uvicorn** in development, with the `-reload` flag so saving a Python file restarts the server. In production a process manager (gunicorn with the `uvicorn.workers.UvicornWorker` class) would be more appropriate, but the thesis

scope does not include production deployment.

## 3.3 SQLAlchemy 2 (Asynchronous)

Persistence in Ovidius NIDS goes through **SQLAlchemy 2** [1], the major release that unified the older session and core APIs around the async-capable `AsyncSession`. The async story matters here because the dispatcher and every API route are coroutines, and mixing synchronous SQLAlchemy calls into that pipeline would either block the event loop or force a thread pool I do not want to maintain.

The data model is declared with the modern typed-mapping syntax in `backend/app/models/`, one file per table. Each model uses `Mapped[T]` annotations for SQLAlchemy 2 type inference. For example, the `Alert` table is declared with fields like:

```
class Alert(Base):
    __tablename__ = "alerts"

    id: Mapped[int] = mapped_column(primary_key=True)
    ts: Mapped[datetime] = mapped_column(default=lambda: datetime.now(timezone.utc))
    severity: Mapped[str] = mapped_column(String(16))
    category: Mapped[str] = mapped_column(String(64))
    detection_method: Mapped[str] = mapped_column(String(16))
    ml_score: Mapped[float|None] = mapped_column(Float, nullable=True)
    # ... and so on
```

**Listing 3.1:** Excerpt from backend/app/models/alert.py

The asynchronous engine is created once at boot in `backend/app/core/database.py` and wrapped in an `async_sessionmaker` so every request can pull a fresh session. The `get_db` dependency yields the session and closes it in a `finally` block, which guarantees no session leaks even if an endpoint raises.

## 3.4 aiosqlite and SQLite

Storage lives in **SQLite** [10] talked to through the asynchronous **aiosqlite** driver. Picking SQLite for a SIEM-style application is the unusual call here — the textbook answer is PostgreSQL — and I want to be honest about why I made it. One overriding reason: deployment cost. SQLite is a single file on disk. There is no separate server to install, no `pg_hba.conf` to configure, no backup cron, no port to expose. For the actual

deployment target of this thesis (a laptop running an academic demo, plus the future small-network template the codebase implies) the trade-off is correct. For a production SOC swallowing thousands of alerts a minute SQLite would be the wrong answer, and the SQLAlchemy abstraction makes the swap to Postgres a one-line URL change.

SQLite has one architectural quirk worth mentioning: it serialises writes (only one writer can hold the database lock at a time). Under the synthetic traffic load used during development (24–40 packets per second, with occasional attack bursts of 60–90 packets) this is not a bottleneck. Even under the heaviest synthetic scenario (the ICMP flood from the Attack Console, which generates upwards of ninety packets per burst), the writes are batched at the alert level rather than the packet level, so the load on SQLite is still measured in tens of writes per second.

## 3.5 Pydantic v2

The validation boundary between JSON on the wire and Python objects in the route handlers is owned by **Pydantic v2** [6]. Each request and response schema lives in `backend/app/schemas/`, declared as a typed class — one model per resource, named after it (`AlertOut`, `RuleIn`, `MLTrainRequest`). The framework checks every payload at runtime and either coerces it into the model or replies with a 422 carrying a precise error trail down to the offending field, which removed about half the error-handling boilerplate I would otherwise have written by hand.

A small but important Pydantic detail: response models with `model_config = ConfigDict(from_attributes=True)` can be built from SQLAlchemy ORM instances directly. The endpoint code reads:

```
@router.get("/alerts/{alert_id}", response_model=AlertOut)
async def get_alert(alert_id: int, db: AsyncSession = Depends(get_db)) -> AlertOut:
    row = (await db.execute(select(Alert).where(Alert.id == alert_id))).
        scalar_one_or_none()
    if not row:
        raise HTTPException(404, "Alert not found")
    return AlertOut.model_validate(row)
```

which is essentially as concise as the equivalent operation in any modern Python ORM-based framework.

## 3.6 scikit-learn

The machine-learning side of the project sits on top of **scikit-learn** [18]. I touch a small slice of the library: `RandomForestClassifier` and `IsolationForest` from `sklearn.ensemble`, plus four metrics (`accuracy_score`, `precision_score`, `recall_score`, `f1_score`) from `sklearn.metrics`. The training data is synthesised inside the detector itself in `backend/app/engine/ml_detector.py` with numpy and a seeded random generator (`numpy.random.default_rng(42)`) so that the same training session reproduces the same model bit-for-bit on any machine that runs the project. Trained artefacts are pickled with `joblib.dump` into `backend/data/models/*.joblib` and loaded lazily by `joblib.load` when the operator activates them from the UI.

The two algorithms share a common `score(pkt) -> (label, score)` interface, so the dispatcher does not need to know which one is running. The `is_anomaly` flag on the detector state distinguishes the two cases at inference time: for Random Forest the score is `predict_proba()[0][1]`; for Isolation Forest it is the negative of `decision_function()[0]`.

## 3.7 Scapy

For live packet capture I use **Scapy** [2]. The library wraps the platform's raw-socket facility (libpcap on Linux and macOS, Npcap on Windows) and presents each captured frame as a stack of Python objects, one per protocol layer. The project hits this surface in two places.

First, the `LiveSniffer` class in `backend/app/engine/sniffer.py` calls `sniff()` on a background thread and converts each captured frame to the internal `Packet` dataclass. The conversion extracts source and destination IPs from the IP layer, source and destination ports from the TCP or UDP layer (or zero for ICMP), TCP flags as a comma-separated string (`"S,A"` rather than the raw integer `0x12`), and the payload bytes from the `Raw` layer if present.

Second, the project ships an optional BPF filter. The `SNIFFER_BPF_FILTER` environment variable is applied at capture time by `sniff()` so uninteresting traffic is dropped before reaching the dispatcher. The expression follows the standard tcpdump syntax (e.g. `"tcp or icmp"` or `"port not 5353"`) and can be left empty to capture everything.

Scapy's dependency on a platform packet driver is the source of most deployment

friction. On Linux the kernel exposes raw sockets to processes that have the `CAP_NET_RAW` capability, so a non-root user can capture if the binary is granted the capability with `setcap`. On Windows the user must install Npcap (a five-megabyte installer from npcap.com) and run the launcher with administrator privileges. The project handles the missing-Scapy case gracefully: if the import fails, the `TrafficSimulator` is used unconditionally regardless of the `SIMULATE_TRAFFIC` setting.

## 3.8 NumPy

**NumPy** [9] is the numerical library underlying scikit-learn. Ovidius NIDS uses NumPy directly only in the feature extraction path inside the ML detector, where each captured `Packet` is converted to a six-dimensional `np.ndarray` before being passed to `model.predict()`. The arrays are constructed with `dtype=float` so that the dispatcher does not incur a copy when scikit-learn validates the input shape.

## 3.9 React 18

The analyst console is a **React 18** [15] single-page application, navigated by React Router. State lives in two places: React Context (authentication and the two live WebSocket streams, both mounted at the Layout level) and component-local `useState` hooks for everything else. No Redux, no Zustand, no MobX — the data shape is small enough that adding a state-management library would have been weight for its own sake.

I picked React over Vue, Svelte and Angular for an unglamorous reason: familiarity. The data flow here is the canonical SPA loop (server pushes JSON, client renders it), and any of the four frameworks would have produced an equivalent dashboard. What tipped the decision was the depth of the React ecosystem around the two pieces I knew I needed — a charting library and an icon set. Both happen to be Recharts and lucide-react below, and both were one `npm install` away.

One React 18 feature the project actually leans on is concurrent rendering through Suspense boundaries on the main routes. With that in place the console stays responsive to clicks and keystrokes even while a slow `fetch` is in flight against the backend, which became visible the first time I hit `/api/alerts?limit=500` on a hot SQLite database.

## 3.10 Vite

The frontend's dev server and production bundler is **Vite** [31], written by Evan You (the same author behind Vue). For new React projects Vite has been steadily displacing the older Create-React-App + webpack pair, and the reason is its dev-time architecture: instead of bundling the whole app on every save, Vite serves each module as a separate HTTP request and lets the browser's own ES-module loader chase the dependency graph. The practical consequence is hot reloads under a hundred milliseconds even on a project the size of the analyst console. I noticed the difference the first time I came back from working on a webpack project — it stops feeling like "save and wait" and starts feeling like "save and look".

The Vite configuration for this project is minimal: a single `vite.config.js` file with the React plugin and a proxy entry that forwards `/api/` and `/ws/` to the FastAPI backend on port 8000 during development. Production builds emit a static `dist/` directory which could be served by any static-file web server.

## 3.11 TailwindCSS

The visual identity of the console is delivered by **TailwindCSS** [29]. The design language is narrow on purpose: a deep navy background (`0A1220`), a cyan primary accent (`22D3EE`), and three semantic highlights — fuchsia for ML-related state, amber for warnings and attack-console interactions, and red for critical alerts. Tailwind's utility-class approach lets me compose that palette directly in the JSX markup; there is no parallel `*.module.css` hierarchy to keep in sync with the component tree.

I extended the default palette in `tailwind.config.js` with three custom navy shades (`navy-850`, `navy-900`, `navy-950`) so the components can refer to named tokens instead of raw hex strings. The final production stylesheet weighs about twenty kilobytes gzipped — small enough that the console feels instantaneous over any network worth measuring.

## 3.12 Recharts

**Recharts** [22] is the React-native charting library that draws the four charts on the dashboard: the live packet-rate area chart, the severity distribution pie, the alert timeline bar chart and the detection-method pie. The library renders SVG under the

hood and supports declarative composition — a chart is a tree of React components rather than a single black-box configuration object, which makes it natural to override colours and tooltip styling per-chart.

A small subtlety I had to resolve was the default chart animation, which caused the live packet-rate area to wiggle every second as Recharts interpolated between the old and new data series. Setting `isAnimationActive={false}` on the `Area` component, plus `type="linear"` and `dot={false}`, produced the stable left-shifting curve I wanted.

## 3.13 lucide-react

**lucide-react** is the icon library — a React port of the Lucide icon set, itself a community fork of the original Feather Icons. The icons are SVG-based, tree-shakeable (only the icons actually imported are included in the final bundle) and visually consistent because they share a single stroke style. Every button, chip and badge in the console uses lucide icons.

## 3.14 date-fns

**date-fns** [13] formats the timestamps shown in the UI. It was preferred over Moment.js (now in maintenance mode) and Day.js (a Moment-compatible alternative) because date-fns is purely functional, tree-shakeable, and the single `format` import is enough for every place a timestamp appears.

## 3.15 WebSocket Protocol

**WebSocket** [7] (RFC 6455) is the bidirectional streaming protocol that delivers live updates from the backend to the analyst console. The two endpoints are `/ws/alerts` (each new alert is pushed as a single JSON message) and `/ws/traffic` (packet metadata batched in groups of up to 50 every 500 ms to keep the dashboard chart fluid without overwhelming the network).

Authentication is handled through a token in the query string because browser WebSocket clients cannot set custom request headers — this is a deliberate restriction of the standard. The token is the same JWT used by the REST endpoints, and the `ws/alerts.py` handler decodes it before subscribing the connection to the alert bus.

## 3.16 JWT and bcrypt

The authentication subsystem is built on two pieces. **JSON Web Tokens (JWT)** [11] encode the signed-in user's identity and role; the backend signs them with the HS256 symmetric algorithm using the secret stored in `backend/.env`. **bcrypt** [19] hashes user passwords before they are stored. I deliberately removed the older `passlib` library from the dependencies because passlib's maintenance has lapsed and it does not work cleanly with the modern bcrypt 4.x and 5.x releases that probe the `__about__._-_version__` attribute. Calling `bcrypt.hashpw` and `bcrypt.checkpw` directly from `backend/app/core/security.py` avoids that whole problem.

## 3.17 Joblib

**joblib** [28] is the persistence layer for the trained machine-learning models. It is a thin wrapper around pickle with optimised handling for large NumPy arrays — the kind of data structure a Random Forest with 200 trees produces. Each trained model is saved to `backend/data/models/` with a name that includes the timestamp, so re-training never overwrites a previous artefact.

## 3.18 PyYAML

**PyYAML** [25] parses the default rule file. The seed rules live in `backend/data/rules/default_rules.yaml` and are loaded into the database the first time the application starts. The format is straightforward enough that an operator can edit the file by hand without learning the JSON match specification — the YAML schema and the database match column are identical apart from the surface syntax.

## 3.19 Uvicorn

**Uvicorn** [5] is the ASGI server that runs the FastAPI application. It is the reference implementation of the ASGI specification, written by Tom Christie, and has become the default choice for asynchronous Python web services. The launcher invokes it as `python -m uvicorn app.main:app -host 0.0.0.0 -port 8000 -reload`; the `-reload` flag watches the file system and restarts the server when source files change, which is invaluable during development.

## 3.20 Node.js and npm

**Node.js** is the JavaScript runtime that powers the Vite development server and the build pipeline. The project pins no specific Node version but requires 18 LTS or newer, which is what the major distributions and the official Node installer ship today. **npm** is the package manager that installs the frontend dependencies; it is bundled with the Node installer, so a user who follows the standard installation procedure has both tools available with no extra configuration.

## 3.21 Operating System Considerations

The project targets three operating systems: Windows 10/11, macOS 12+, and any reasonably recent Linux distribution. The cross-platform launcher (`run.py`) detects the host operating system via `platform.system()` and adjusts the venv path (`Scripts/python.exe` on Windows versus `bin/python` on Unix), the signal handling (`CTRL_BREAK_EVENT` on Windows versus `SIGTERM` on Unix) and the dependency installation procedure. The two wrappers (`run.bat` for Windows, `run.sh` for Unix) handle the shell-specific quirks — mostly around how to invoke Python with the right version — and then delegate to `run.py` for the actual logic.

# Chapter 4

# Related Work

The intrusion detection landscape is dense. To position Ovidius NIDS honestly I divided the comparison into three groups: production-grade open-source NIDS engines, commercial SIEM platforms and the academic machine-learning IDS literature. Each section ends with a short note on how my project differs.

## 4.1 Snort

I'll start with the tool everyone benchmarks against. **Snort** [23] arrived in 1998 from Martin Roesch as the open-source answer to expensive commercial NIDS appliances of the late 1990s. Almost three decades later it is still under active development; the current C++ rewrite (Snort 3, 2021) replaced the original C engine that Roesch wrote, but the detection philosophy hasn't shifted: rules in, alerts out, no UI between them.

What people actually buy into when they pick Snort is the rules ecosystem rather than the engine. Cisco's Talos team (Cisco bought Sourcefire, the company behind Snort, back in 2013) ships a daily Subscriber rule set for paying customers and a weekly Community one for free users. Proofpoint runs Emerging Threats, a third-party project that distributes thousands of additional rules at no charge. A real-world Snort deployment is essentially a subscription to detection logic written by other people — the engine's biggest strength on day one, and the recurring tax on day two when the rules need pruning, tuning and retiring.

On the engineering side Snort was built for line-rate inspection. With the right NIC and config it'll happily process multiple gigabits per second of inline traffic. The cost of that throughput is an interface aimed squarely at terminal-comfortable operators. There's no analyst console in the box. Snort writes unified2 binary logs that some downstream pipeline (Barnyard2 typically, then a SIEM, then a dashboard) has to chew through before anyone sees an alert. Operators who want a UI pick between SecurityOnion (an entire Linux distribution that bundles Snort with half a dozen other tools), the commercial Snort Cloud offering from Cisco, or some homegrown stack.

The shape of Ovidius NIDS's signature engine borrows from Snort, but two things differ. The match specification is JSON instead of a custom DSL, and it's editable from the UI rather than a text file in `/etc/snort/`. And the analyst console ships in the same binary as the detection engine instead of being somebody else's problem.

## 4.2 Suricata

The number-two engine in the open-source NIDS world is **Suricata** [16]. The Open Information Security Foundation (OISF) started it in 2009 with a clean-slate multi-threaded rewrite of the same ideas Snort had pioneered a decade earlier. The trick the OISF pulled was making Suricata read Snort rules (with a few extensions of its own), so the rule ecosystem is shared — if you write detection for one engine you get the other almost for free.

Three things make Suricata interesting from an engineering angle. First, threads: it scales close to linearly with CPU cores out of the box, where Snort needed serious tuning to do the same. Second, protocol parsing: HTTP, TLS, SSH, SMB and DNS are decoded before the matching happens, so a rule can fire on parsed fields (`tls.sni`, `http.uri`) rather than raw payload bytes. Third, the Eve log format — one JSON object per event, structured, easy for downstream SIEMs to ingest. Compared to Snort's unified2 binary blob, that's a real quality-of-life upgrade.

Like Snort, though, Suricata is a daemon. OISF publishes Scirius on the side for rule management and a separate community project EveBox for alert browsing, but the engine itself ships without an analyst-facing UI.

I borrowed Suricata's structured-logging idea — JSON in, JSON out, no binary intermediates — but stopped short of the protocol-aware matching it's famous for. The rule vocabulary in Ovidius NIDS works at the TCP/UDP/ICMP level on headers and payloads, which covers the validation scenarios (port scans, brute-force attempts, simple

floods, content-based exploits). What it can't do is the parser-level matching that's needed for HTTP request smuggling, TLS-fingerprint analysis or DNS tunnelling — those would require pulling in a stateful protocol parser layer, which is its own project.

## 4.3 Zeek

**Zeek** [17] (the project everyone still calls Bro, two decades after Vern Paxson started it under that name) belongs to a different category altogether. Snort and Suricata are pattern matchers. Zeek is a programming environment for the network. Operators write scripts in Zeek's own scripting language and hang them on events the engine emits — `connection_established`, `http_request`, `dns_request`, and a long list of others — and the scripts decide what to log. Out of the box the engine doesn't produce "alerts" in the Snort sense. It produces structured log streams (`conn.log`, `http.log`, `dns.log`, and dozens more) and leaves the detection logic to whatever the operator scripts or to a downstream SIEM.

That design is why threat-hunters love it. If your job is asking exploratory questions of the traffic record — "which hosts talked to this domain in the last 30 days, on what ports, with what user-agent strings?" — Zeek is the right answer. Its logs are also the de-facto input format for academic ML-IDS research. CICIDS2017 and IOT-23, the datasets every recent paper benchmarks against, ship Zeek log bundles alongside the raw PCAP.

Ovidius NIDS doesn't try to compete in this space. The audience this project targets isn't the researcher who wants to script arbitrary queries over weeks of traffic. It's the operator who wants the console to say "alert fired, here it is, here are the packets that produced it, what do you want to do". The two tools could live next to each other in a real deployment — Zeek doing the forensic logging, Ovidius NIDS doing the real-time alerting — without overlap.

## 4.4 Wazuh

**Wazuh** [30] sits a tier up the stack. It grew out of OSSEC — the long-running host-based IDS — and added a central manager, a Kibana-based console and a pile of integrations with external network engines. The detection that Wazuh itself does is primarily host-side: log analysis, file integrity monitoring, vulnerability assessment, audit-rule matching. For network-layer detection it relies on a sidecar that ingests

alerts from Snort or Suricata — Wazuh aggregates and dashboards them, but it doesn't generate them.

What Wazuh ships is a full operations console: rule management, incident queues, agent administration, configurable dashboards on top of Elasticsearch (or its OpenSearch fork). All of it is production-grade. The price you pay is infrastructure footprint. A minimal Wazuh single-node deployment wants several gigabytes of RAM just to keep the search index responsive, and the recommended cluster setup runs three to five nodes.

Ovidius NIDS differs from Wazuh in the same way a single-binary SQLite app differs from a clustered PostgreSQL deployment: by an order of magnitude in setup complexity. Wazuh would be the right choice for an enterprise. Ovidius NIDS is the right choice for a laptop demo or a small-network deployment where the SIEM is allowed to be one process and one database file.

## 4.5 Splunk Enterprise Security

The benchmark in the commercial SIEM market is **Splunk Enterprise Security**. Architecturally it works much like Wazuh — agents on hosts feed events into a central indexer; analysts query that indexer from a web console; saved correlation searches roll up matched events into "notable events" that feed an incident queue. What separates Splunk is the polish of the analyst experience. When IBM QRadar or Microsoft Sentinel get reviewed in trade press, the comparison is almost always against Splunk.

The cost of that polish is the other reason Splunk gets talked about. Licensing is volume-priced — you pay per gigabyte of data ingested per day — and even a single-instance ES deployment lands in five-figure annual budgets. There is a free Splunk Free tier, but it caps ingest at 500 MB per day and doesn't include the Enterprise Security app at all.

So Ovidius NIDS is not in the same market. It sits at the opposite end of the spectrum: free, self-hosted, single binary, for environments where the budget is zero and the deployment needs to take minutes instead of weeks.

## 4.6 Academic Machine-Learning IDS

The published machine-learning IDS literature is, as discussed in Section 2.3, vast. The bulk of it consists of papers that propose a particular classifier architecture (a deep neural network, a gradient-boosted ensemble, an autoencoder, a transformer) and evaluate it on a standardised dataset. The two most-cited datasets in recent work are **NSL-KDD** [27] (a cleaned version of the 1999 KDD Cup dataset) and **CICIDS2017** [24] (a more recent dataset from the Canadian Institute for Cybersecurity).

The Sommer and Paxson critique [26] is now fifteen years old but remains relevant. They argue, persuasively, that the closed-world assumption of standard machine-learning evaluation does not transfer to operational network security. A classifier that scores 0.99 F1 on NSL-KDD is not necessarily a useful production tool, because the distribution of normal traffic in NSL-KDD is not the distribution of normal traffic on any real network. Sommer and Paxson recommend three concrete safeguards: prefer anomaly detection over misuse detection for the ML component, give the operator a clear explanation of why each alert fired, and keep the ML engine in a supporting role behind a signature engine.

Ovidius NIDS explicitly adopts all three recommendations. Isolation Forest is offered as a first-class option for operators who lack labelled attack data; every alert is tagged with its detection method; and the hybrid category is the high-confidence one, implicitly demoting the ml-only category to a “second opinion” role.

## 4.7 Positioning of This Work

The position Ovidius NIDS occupies is most easily summarised in a feature matrix; Table 4.1 reproduces the comparison from the presentation deck.

The differentiators that matter for a thesis-scale project, and that no other tool in the table provides simultaneously, are:

- Detection, ML, forensics and UI in a single binary.
- Per-alert packet capture as a core feature rather than a bolt-on integration.
- One-click deployment with no infrastructure dependencies.
- Hybrid alert labelling that distinguishes signature, ML and cooperative detection visually.

**Table 4.1:** Feature comparison of Ovidius NIDS against related tools. "✓" indicates the feature is present out of the box, "∼" indicates partial or plugin-based support, "–" indicates the feature is outside the tool's scope, and "×" indicates the feature is absent.

| | Snort | Suricata | Zeek | Wazuh | Splunk ES | **Ovidius NIDS** |
|---|---|---|---|---|---|---|
| Signature engine | ✓ | ✓ | ∼ | ∼ | ∼ | ✓ |
| ML engine | × | × | × | ∼ | ✓ | ✓ |
| Hybrid label | – | – | – | × | × | ✓ |
| Built-in analyst UI | × | × | × | ✓ | ✓ | ✓ |
| Per-alert PCAP | × | × | × | × | × | ✓ |
| Real-time stream UI | × | × | × | ∼ | ✓ | ✓ |
| Editable rules from UI | × | × | × | ✓ | ✓ | ✓ |
| One-click deploy (Win/Mac/Linux) | × | × | × | × | × | ✓ |
| Self-hostable | ✓ | ✓ | ✓ | ✓ | ∼ | ✓ |
| Free / open-source | ✓ | ✓ | ✓ | ✓ | × | ✓ |

The trade-off is scale. Ovidius NIDS is not built to inspect multi-gigabit traffic on a forty-core appliance; it is built to run on a laptop and demonstrate the cooperation of the three engines on real packets. That is the niche, and the comparison table makes it explicit.

# Chapter 5

# Design and Implementation

This chapter walks through every component of Ovidius NIDS with references to the actual source files. The order mirrors the direction of data flow: a packet enters at the capture layer, is processed by the dispatcher, is evaluated by the two detection engines, may produce one or more alerts, and is finally persisted and broadcast. The frontend, the API, the database schema and the launcher are covered in their own sections after the core pipeline.

## 5.1 General Architecture

### 5.1.1 The Two-Tier Split

The system is split into a Python backend and a JavaScript frontend. The split is the conventional one for modern web applications: the backend owns the data, the business logic and the detection engines; the frontend owns the rendering and the user interaction. They communicate over HTTP for queries and over WebSocket for live updates.

Both halves live in the same repository (`Licenta_NIDS/`) but are managed by separate package managers. The backend uses pip through a virtual environment created at `backend/.venv/`; the frontend uses npm through the standard `frontend/node_modules/`

directory. The two installations are orchestrated by the cross-platform launcher described in Section 5.13.

### 5.1.2 Packet Life Cycle

The path a packet takes from arrival to UI notification is short enough to enumerate in eight steps:

1. The packet enters either the Scapy `LiveSniffer` on a real network interface, or the in-process `TrafficSimulator` if simulation mode is enabled. Both paths converge on the same internal `Packet` dataclass.
2. The packet handler — a coroutine bound at startup — invokes `Dispatcher.handle()` via `asyncio.run_coroutine_threadsafe` (in the case of the Scapy thread) or via a direct await (in the case of the simulator).
3. `Dispatcher.handle()` updates the packet counter and the sliding-window packets-per-second meter, appends the packet to the in-memory ring buffer of size 200, and publishes a compact summary on the traffic event bus.
4. Every fifth packet is sampled into a `TrafficEvent` row in the database. The sampling ratio is configurable (`TRAFFIC_SAMPLE` constant in `dispatcher.py`); the default keeps the events table small enough for demo use.
5. `Dispatcher._evaluate` runs the signature engine and, if a model is loaded, the ML detector. The two verdicts are merged into zero, one or several alert candidates, each carrying an explicit `detection_method` field.
6. Each candidate becomes an `Alert` row. The dispatcher writes the alert, generates a per-alert JSONL file with the last 50 packets from the ring buffer, writes a `PcapRecord` row indexing the file, and commits.
7. The dispatcher publishes the alert payload on the alert event bus. Any WebSocket clients subscribed to `/ws/alerts` receive the payload immediately.
8. The frontend's `useLiveAlerts` hook prepends the new alert to its in-memory list, which causes the dashboard live feed and the alerts page to re-render.

Step (3) and step (4) run concurrently with step (5) only at the asyncio level — there is no thread-level parallelism inside the dispatcher beyond the Scapy capture thread itself. The choice is deliberate: a packet-level lock would have added complexity for no measurable benefit at the throughput the validation workload generates.




**Figure 5.1:** High-level architecture: a packet enters at the source (a NIC via Scapy, or the in-process simulator), is captured, handed to the central dispatcher, evaluated by both the signature and ML engines, and exits as a database row, a JSONL file on disk and a WebSocket message to the browser.

## 5.2 Capture Layer

The capture layer is the boundary between the outside world and the detection pipeline. It is implemented as two interchangeable classes in `backend/app/engine/sniffer.py`, both honouring the same async callback contract:

```
1 PacketHandler = Callable[[Packet], Awaitable[None]]
```

**Listing 5.1:** The capture-layer interface

### 5.2.1 LiveSniffer (Scapy)

`LiveSniffer` wraps Scapy's blocking `sniff()` call on a dedicated background thread. The thread is started with `daemon=True` so it does not block process shutdown, and it posts each parsed packet back to the FastAPI asyncio event loop via `asyncio.run_coroutine_threadsafe(self.handler(pkt), self._loop)`. The conversion from Scapy's nested layered structure to the internal `Packet` dataclass happens in `_to_packet`: the IP layer gives source and destination addresses, the TCP or UDP layer gives ports, the TCP flags are extracted from the `scapy_pkt[TCP].flags` object and normalised to a comma-separated string (`"S,A"` rather than the bitmask `0x12`) so the signature engine can split them consistently regardless of whether the source was Scapy or the simulator.

The class accepts a BPF filter via the constructor and forwards it to `sniff(filter=...)`, which means uninteresting traffic is dropped at the kernel boundary before reaching Python. Default is empty (capture everything).

### 5.2.2 TrafficSimulator

`TrafficSimulator` is the in-process packet generator. It runs as an asyncio task started by the FastAPI lifespan handler and emits two kinds of traffic: a benign baseline and on-demand attacks.

The benign baseline is controlled by the `SIM_BENIGN_TRAFFIC` environment variable. When enabled, the simulator emits 6–10 benign packets every 250 ms (yielding a steady 24–40 packets per second). Each benign packet has a random source IP from the LAN client pool (`10.0.0.20` through `10.0.0.59`), a destination of the configured defended host (`DEFENDED_HOST_IP`, default `10.0.0.10`), a random ephemeral source port, and one of the four common destination ports (80, 443, 53, 22). The benign baseline is disabled by default so the dashboard's live packet rate chart stays flat until the operator deliberately enables traffic — this makes the cause and effect of attack injection visually obvious during demonstration.

The on-demand attacks are exposed through six factory methods, one per attack family:

- `_nmap_scan(attacker)` emits 25–40 TCP SYN packets sweeping 14 common service ports.
- `_ssh_brute(attacker)` emits 12–20 SYNs to port 22 from a single source.
- `_icmp_flood(attacker)` emits 55–90 ICMP echo packets.
- `_sqli(attacker)` emits one TCP packet containing the classic `' OR '1'='1` payload in an HTTP GET.
- `_cmd_inject(attacker)` emits one HTTP POST with shell metacharacters in the body.
- `_reverse_shell(attacker)` emits one TCP packet containing a `/bin/sh` redirection pattern.

All six are routed at the configured defended host. The choice of attacker IP comes from a small pool (`45.77.12.88`, `185.220.101.5`, `198.51.100.23`); the pool can be overridden by the API caller through the optional `attacker` parameter on the `fire_attack` method.

### 5.2.3 Mode Selection

The FastAPI lifespan handler in `backend/app/main.py` reads the configuration and starts exactly one of the two capture sources. The selection logic is:

```
if settings.simulate_traffic or not settings.sniffer_interface:
    sim = TrafficSimulator(handler=dispatcher.handle, ...)
    sim.start()
    app.state.simulator = sim
else:
    try:
        live = LiveSniffer(...)
        live.start(asyncio.get_running_loop())
        app.state.sniffer = live
    except Exception as exc:
        log.warning("Live sniffer failed (%s). Falling back to simulator.", exc)
        sim = TrafficSimulator(handler=dispatcher.handle, ...)
        sim.start()
        app.state.simulator = sim
```

The fallback is important: if the operator forgets to install Npcap on Windows, the application does not crash — it logs a warning and starts the simulator, so the dashboard always has data.

## 5.3 The Dispatcher

The `Dispatcher` class in `backend/app/engine/dispatcher.py` is the central coordination point. It is instantiated once at startup by `build_dispatcher()` and stored as the module-level `dispatcher` singleton plus on `app.state.dispatcher`.

The constructor takes the path to the PCAP output directory and a flag indicating whether ML scoring is enabled. The instance holds the packet counter, the alert counter, the ring buffer (`deque(maxlen=200)`) and the sliding-window PPS tracker. The core entry point is `handle()`, shown in Listing 5.2.

```
async def handle(self, pkt: Packet) -> None:
    self._pkt_count += 1
    self._ring.append(pkt)
    self._tick_pps()

    await self._publish_traffic(pkt)
    await self._maybe_persist_event(pkt)

```

```
9       alerts = self._evaluate(pkt)
10      if alerts:
11          await self._persist_and_broadcast(alerts)
```

**Listing 5.2:** Dispatcher entry point

The function reads like a flow diagram. Each helper method is small (10–30 lines), pure with respect to its inputs, and easy to unit-test. The same packet is observed by three independent consumers (metrics, sampling, evaluation) without any of them needing to know about the others.

## 5.4 Signature Engine

The signature engine lives in `backend/app/engine/signatures.py`. It evaluates each packet against the in-memory list of compiled rules and returns zero or more alert candidates.

### 5.4.1 Match Specification

A rule's `match` field is a JSON object with up to nine keys:

- `protocol` — `"tcp"`, `"udp"`, `"icmp"` or `"any"`.
- `src_ip` and `dst_ip` — single IP, CIDR notation, or `"any"`.
- `src_port` and `dst_port` — single port, range string `"80-8080"`, list, or `"any"`.
- `flags` — list of TCP flag letters that must all be set (e.g. `["S"]` for SYN-only, `["S","A"]` for SYN-ACK).
- `content` — a case-insensitive substring search over the payload bytes.
- `regex` — a Python regular expression evaluated against the payload decoded as UTF-8 with replacement.
- `min_length` — a minimum frame length in bytes.
- `threshold` — a sub-object with `count`, `seconds` and `track` fields turning the rule into a sliding-window counter.

The default rule set is seeded from `backend/data/rules/default_rules.yaml`. The ten bundled rules cover the six attack families used in the validation scenario plus a few demonstration rules (suspicious TLS SNI, plaintext Telnet, oversized UDP). The full YAML for the Nmap rule illustrates the format:

```yaml
- name: "Nmap TCP SYN scan"
  description: "High rate of TCP SYN packets from a single host (port scan signature
      )."
  category: "Reconnaissance"
  severity: "medium"
  action: "alert"
  match:
    protocol: "tcp"
    flags: ["S"]
    threshold:
      count: 8
      seconds: 5
      track: "src_ip"
```

**Listing 5.3:** The Nmap scan rule, as stored in the seed YAML

### 5.4.2 Threshold Tracking

The threshold logic is implemented by a small helper class, `_ThresholdTracker`, which maintains a sliding window of event timestamps keyed by the tuple (*rule id*, *track value*). When a rule with a threshold fires, the tracker appends the current monotonic timestamp to the appropriate deque, drops any entries older than the configured window, and returns `True` if the deque length has reached the configured count.

The tracker uses `time.monotonic()` rather than wall-clock time, so that NTP adjustments do not perturb the window. For the Nmap rule with `count=8, seconds=5`, an attacker who sends 8 SYNs within any five-second period will trigger an alert; an attacker who paces the scan with a 700-millisecond delay between probes will not.

### 5.4.3 Evaluate Loop

The `evaluate()` method iterates the compiled rules and applies each predicate in order:

1. Protocol match (or `any`).
2. Source IP / CIDR match.

3. Destination IP / CIDR match.
4. Source port match (single, range, list).
5. Destination port match.
6. TCP flag subset match.
7. Content substring (case-insensitive).
8. Regex search.
9. Minimum frame length.
10. Threshold (if present) — only the last step that actually increments any state.

The order is deliberate: cheap checks first, expensive ones last. A rule that does not match the protocol is dismissed in a single string comparison; the regex check, which is the most expensive, only runs for rules where all the cheaper predicates already passed.

## 5.5 Machine-Learning Detector

The ML detector in `backend/app/engine/ml_detector.py` implements the two algorithms behind a common interface.

### 5.5.1 Feature Vector

Each packet is converted into a six-dimensional NumPy array. The features were chosen to be cheap to extract (no payload parsing, no protocol-aware analysis) and to encode the kinds of behaviour the attack catalogue actually exhibits:

1. `src_port` — source port number.
2. `dst_port` — destination port number.
3. `length` — L2 frame length in bytes.
4. `proto_id` — 6 for TCP, 17 for UDP, 1 for ICMP, 0 otherwise. Matches the IANA protocol numbers.
5. `flag_mask` — a bitmask of the TCP flags (FIN=1, SYN=2, RST=4, PSH=8, ACK=16, URG=32). For non-TCP packets it is zero.

6. `inter_arrival_ms` — the gap in milliseconds since the previous packet from the same source IP. Maintained as streaming state in a dictionary inside the detector.

The choice was conservative on purpose. A six-feature vector is small enough to inspect by hand on the Alert Detail page, and it avoids the dataset-leakage trap that more elaborate feature sets fall into [26].

### 5.5.2 Training

The `train_synthetic()` method generates a 4000-sample training set with a 70/30 split between normal and malicious classes. The normal class uses random draws from distributions that mimic the benign baseline (typical ports, moderate frame sizes, exponential inter-arrival around 200 ms). The malicious class uses distributions skewed toward the attack patterns (high ephemeral source ports, attack-favourite destination ports, very small or very large frames, short inter-arrival times).

Both algorithms train on a 75/25 train-test split. `RandomForestClassifier(n_estimators=200, max_depth=12, random_state=42)` fits in roughly one second on a mid-range laptop. `IsolationForest(n_estimators=200, contamination=0.3, random_state=42)` fits in roughly 0.7 seconds and only sees the normal-class samples during training.

Both produce four classification metrics (accuracy, precision, recall, F1) on the held-out test set. Random Forest also exposes `feature_importances_`, which the API surfaces in the `MLModelOut` response and the frontend renders as a Recharts bar chart on the ML Models page.

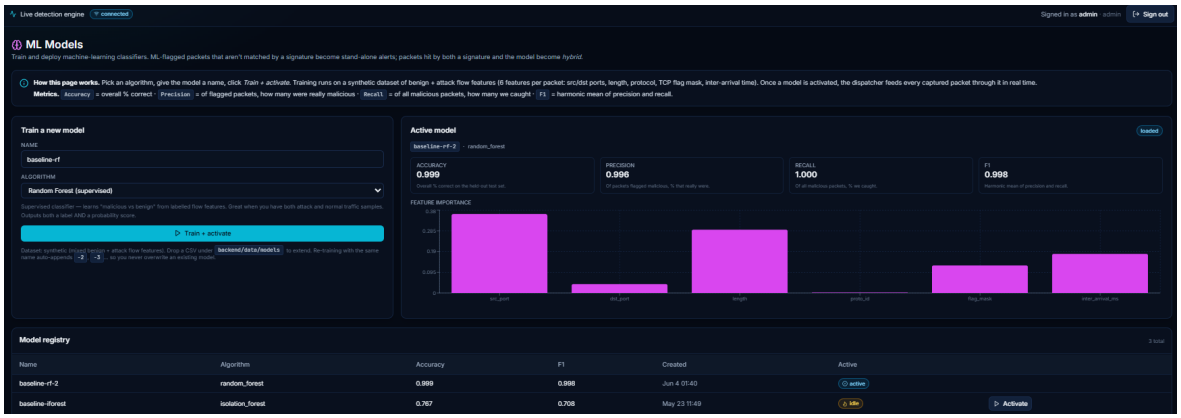



**Figure 5.2:** The ML Models page after training the default Random Forest classifier. The four metric cards report accuracy, precision, recall and F1 on the held-out test split; the bar chart shows the per-feature importance derived from the model's `feature_importances_` attribute, with `src_port` and `length` carrying the most discriminative signal.

### 5.5.3 Inference

The `score(pkt)` method extracts the feature vector for the incoming packet, runs it through the active model and returns a `(label, score)` tuple. For Random Forest the score is `predict_proba()[0][1]`; for Isolation Forest it is the negative of `decision_function()[0]`, so larger values mean more anomalous. The label is 1 (malicious) if the score is at or above the threshold, 0 (benign) otherwise.

The inference path is wrapped in a mutex (`threading.Lock`) so that training and scoring cannot race. In practice training is not concurrent with high-volume scoring since it is triggered by an explicit API call, but the lock makes the code obviously safe.

## 5.6 Hybrid Merge Logic

The merge logic is the heart of the system. It lives in `Dispatcher._evaluate` and is shown in Listing 5.4.

```
def _evaluate(self, pkt: Packet) -> list[dict[str, Any]]:
    signature_hits: list[dict[str, Any]] = list(engine.evaluate(pkt))

    ml_label, ml_score = 0, 0.0
    if self.enable_ml and detector.is_active():
        ml_label, ml_score = detector.score(pkt)

    # ML-only alert: signature engine missed, ML score is high enough
    if ml_label == 1 and ml_score >= ML_ALERT_THRESHOLD and not signature_hits:
        return [self._ml_candidate(pkt, ml_score)]

    # Hybrid upgrade: both engines agreed
    if signature_hits and ml_label == 1:
        for h in signature_hits:
            h["detection_method"] = "hybrid"
            h["ml_score"] = ml_score
            h["ml_label"] = "malicious"

    return signature_hits
```

**Listing 5.4:** Hybrid alert merge logic

The function is twenty lines long and contains every alerting decision the system makes. A signature-only alert is emitted when the signature engine matched and the ML model either disagreed or was not active. An ml-only alert is emitted when the

signature engine missed and the ML model scored above `ML_ALERT_THRESHOLD` (0.8 by default). A hybrid alert is the same row as the signature hit, with the detection method upgraded and the ML score attached.

## 5.7 Forensics Module

The forensics module is implemented by the small helper `write_packet_log` in `backend/app/engine/forensics.py`, plus a few lines in the dispatcher's `_persist_and_broadcast()` method that invoke it.

When an alert is created, the dispatcher takes the last 50 entries from the ring buffer (`list(self._ring)[-SNAPSHOT_TAIL:]`), passes them and the alert ID to `write_packet_log()`, and stores the returned file path on both the `Alert` row and the corresponding `PcapRecord` row. The file is JSON Lines: one packet per line, encoded as a JSON object with the timestamp, the 5-tuple, the TCP flags, the frame length and the first 256 bytes of the payload (decoded as UTF-8 with replacement, since binary protocols do not decode cleanly).

The file name pattern is `alert_{alert_id}_{epoch}.jsonl`. A typical file might be 7 kB for a 50-packet snapshot, which is small enough that the application can keep thousands of them without disk-space concerns.

### 5.7.1 Downloading from the UI

The download endpoint at `GET /api/forensics/alert/{alert_id}/download` returns the file with `FileResponse`. Because the endpoint is protected by `Depends(get_current_user)`, the frontend cannot simply emit a plain `<a href download>` link — the anchor navigation would not carry the JWT in the `Authorization` header and the server would return 401.

The frontend handles this with the `apiDownload(url, filename)` helper in `frontend/src/api/client.js`: it issues a `fetch(url, {headers: {Authorization: 'Bearer ' + token}})`, wraps the response body in a `Blob`, calls `URL.createObjectURL` and triggers a synthetic anchor click. This pattern is the standard workaround for authenticated downloads from a Single-Page Application and resolved the "download.json — File wasn't available on site" error I encountered during development.




**Figure 5.3:** The Forensics page listing the JSONL captures stored on disk. Each row corresponds to one alert and carries a link to the underlying `Alert` record plus a Download button that uses the authenticated `apiDownload` helper. In this run the dispatcher has accumulated several hundred captures, each holding 50 packets at roughly ten kilobytes per file.

## 5.8 REST and WebSocket API

The HTTP surface is organised by FastAPI routers, one per resource type, all mounted in `backend/app/main.py`. The routers are:

- `api/auth.py` — login, current-user, change password.
- `api/alerts.py` — list (with filters), get, update status.
- `api/rules.py` — list, create, update, delete.
- `api/dashboard.py` — aggregated metrics for the dashboard summary.
- `api/forensics.py` — list captures, get packets, download file.
- `api/ml.py` — list models, train, activate, status.
- `api/sim.py` — attack catalogue, fire attack, toggle benign traffic.

Every router (except `auth.py`) declares `dependencies=[Depends(get_current_user)]` at the router level, so every endpoint inherits the authentication requirement without having to repeat it on every route. The dependency is one of the cleanest things FastAPI does: it decodes the JWT, loads the user from the database, and either injects the `User` into the route parameters or raises an HTTPException that becomes a clean 401 response.

The WebSocket endpoints live in `backend/app/ws/alerts.py`. Authentication on these endpoints is awkward because browsers cannot set custom request headers on WebSocket opens; Ovidius NIDS solves this by accepting the JWT as a query parameter (`ws://host/ws/alerts?token=...`). The handler decodes the token before subscribing the connection to the relevant event bus, exactly mirroring the REST behaviour.

The two buses (`alert_bus` and `traffic_bus`, defined in `backend/app/engine/bus.py`) are simple asyncio publish-subscribe primitives: each subscriber holds an asyncio queue, and the bus's `publish()` method writes the event to every subscriber's queue. The `/ws/traffic` endpoint additionally batches and throttles traffic events (up to 50 per batch, flushed every 500 ms) so the chart in the browser does not get bombarded with hundreds of single-packet messages per second under heavy load.

## 5.9 Frontend

The frontend is a React 18 single-page application bundled by Vite. The source tree is small enough to enumerate:

- `src/api/client.js` — HTTP and WebSocket helpers, plus the authenticated `apiDownload` discussed above.
- `src/store/auth.jsx` — the authentication `AuthContext` with login, logout and JWT persistence in `sessionStorage`.
- `src/hooks/useLiveAlerts.js` and `src/hooks/useLiveTraffic.js` — the two WebSocket subscriber hooks, both used by the shared layout.
- `src/components/Layout.jsx` — the sidebar plus main content frame; also where the live WebSocket subscriptions live so they survive page navigation.
- `src/components/Sidebar.jsx`, `components/TopBar.jsx`, `components/SeverityBadge.jsx`, `components/MethodBadge.jsx`, `components/StatCard.jsx`, `components/ConfirmDialog.jsx` — the shared widget set.
- `src/pages/*.jsx` — one file per page (Login, Dashboard, Alerts, AlertDetail, Rules, MLModels, Forensics, AttackConsole, Settings).

### 5.9.1 Authentication and Routing

The router (`App.jsx`) wraps every non-login route in a `RequireAuth` component that reads the auth context and redirects to `/login` if no user is present. The `AuthProvider` restores the token from `sessionStorage` on mount, calls `GET /api/auth/me` to hydrate the user, and unsets the token on a 401 response.

A subtle resilience improvement I added during development is a small retry loop inside the `login` function: transient 5xx responses (which can happen on cold-boot races against the backend's lifespan handler) are retried up to three times with linear backoff before the failure is surfaced to the user. The behaviour eliminates the "Internal server error — press the button again" artefact that briefly affected the first request after a fresh deployment.

### 5.9.2 Live Data Lifting

The two live hooks (`useLiveAlerts` and `useLiveTraffic`) are mounted at the `Layout` level rather than inside the individual pages. The reason is state preservation: hooks mounted inside a page are torn down when the user navigates away, and any ring buffer they maintain is reset. By moving them up to the shared layout, the live data persists across all routes, which fixed the bug where the live packet-rate chart on the dashboard reset to zero every time the user came back from the alerts page.

The shared context exposes a single object with four properties: `alerts`, `connected`, `traffic` and `trafficConnected`. Each page that needs live data reads exactly what it needs via the `useLive()` helper.

### 5.9.3 Pages

The eight pages of the console are described below, each illustrated by a screenshot from a running instance.

**Login** (`Login.jsx`) is a single form bound to the `login` action. Default credentials (`admin` / `admin`) are seeded by the backend on first boot. The deep-navy background and the cyan shield emblem set the visual identity used across the rest of the console; the small "Default credentials" hint at the bottom of the form is removed when the application is deployed in production via the `DEBUG=false` flag.

**Dashboard** (`Dashboard.jsx`) is the operations overview. The top row is four KPI cards (24-hour alert count, open alerts, packet count, packets per second), pulled from

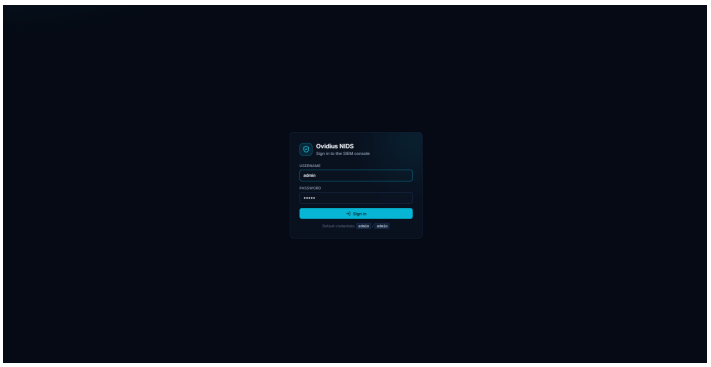


**Figure 5.4:** The Login page. A single form with the OAuth2 password flow against `POST /api/auth/login`; on success the token is written to `sessionStorage` and the user is redirected to the Dashboard.

`/api/dashboard/summary` every five seconds. The middle row is the live packet-rate area chart (from the WebSocket) and a severity distribution pie. The bottom row is the 6-hour alert timeline (10-minute buckets), a detection method pie, and three small panels for top source IPs, top destination ports and the live alert feed. The "Monitoring host" badge in the top right is a single source of truth for which host the operator is watching — it pulls the `defended_host_ip` from `/api/health` and the colour-codes the mode (simulator versus live capture).

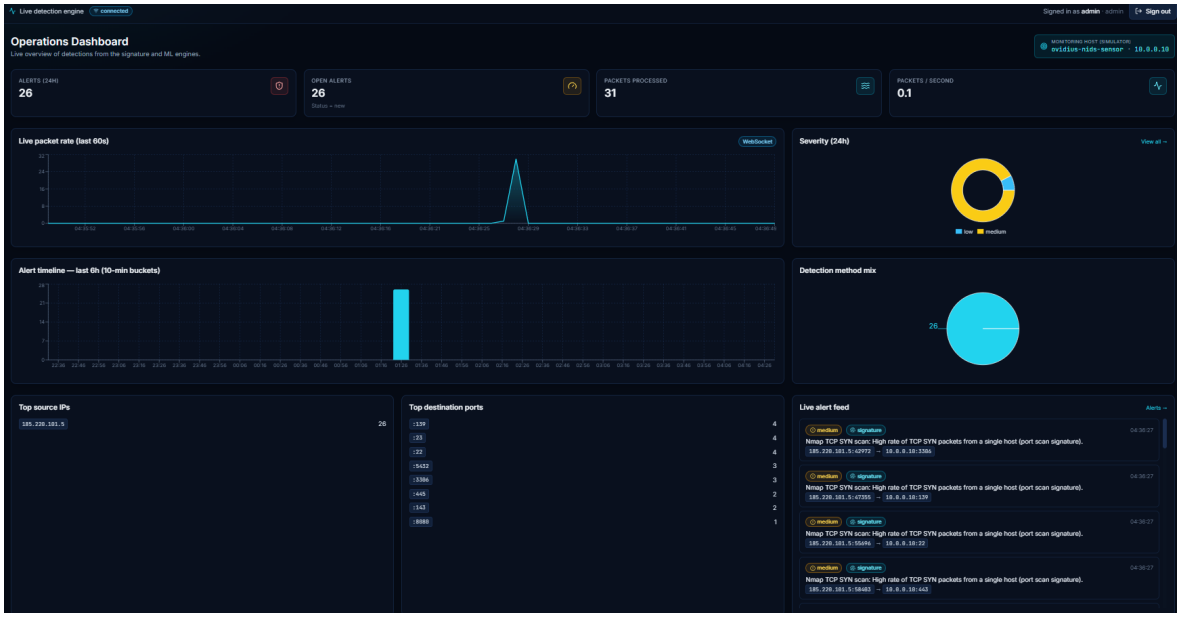



**Figure 5.5:** The Operations Dashboard. The visible spike on the live packet-rate chart is a single attack burst injected through the Attack Console; the 26 alerts produced by that burst show up across the severity pie, the 6-hour timeline (the tall bar around 01:30), the detection-method pie and the live alert feed on the right.

**Alerts** (`Alerts.jsx`) is the triage table. Filters for severity, detection method, status and free-text search work on both the historical rows fetched from `/api/alerts` and

on the live WebSocket stream prepended to the table. Each row has a status dropdown (new / triaged / closed) and an Inspect link to the detail page. The filter widgets are debounced — the search field re-fetches 300 ms after the last keystroke rather than on every character, which is the canonical optimisation for an input-driven query.

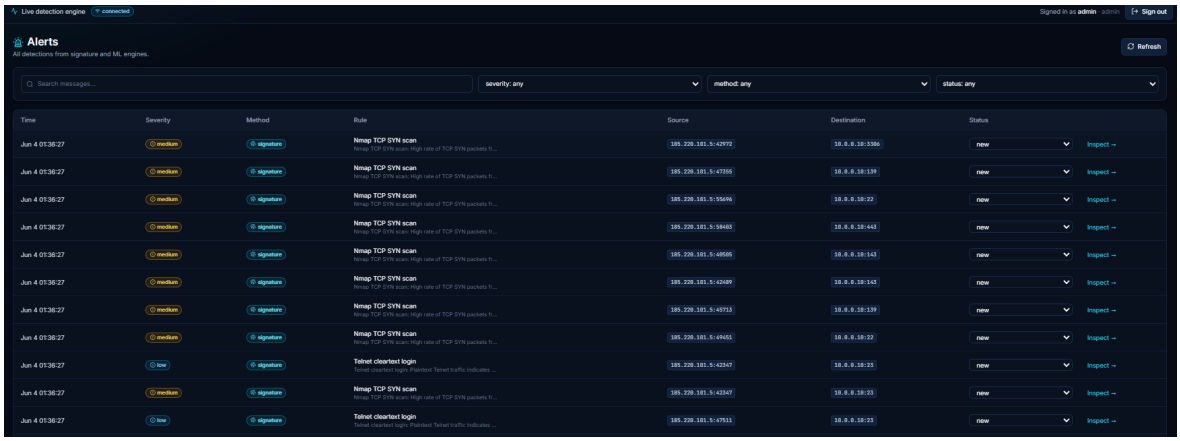



**Figure 5.6:** The Alerts page after a port-scan burst. Every row's detection method is `signature` and the source IP matches the synthetic attacker pool (`185.220.101.5`); the destination IP `10.0.0.10` is the configured defended host. The two `Telnet cleartext login` entries are a different rule that happened to fire on traffic to port 23 from the same attacker burst.

**Alert Detail** (`AlertDetail.jsx`) shows the full record for one alert: severity, detection method, source and destination, payload preview, the path of the JSONL capture, and a table of the captured packets with their flags and lengths. The Download button uses the authenticated `apiDownload` helper.

**Rules** (`Rules.jsx`) is the rule editor. Each rule is a row in a table; clicking Edit opens the rule in a JSON editor where the operator can adjust the match specification. Saving triggers a hot reload of the in-memory rule list on the backend. Deletion is gated by a themed confirmation dialog — a custom component matching the navy-and-cyan look of the rest of the console, rather than the browser's native `window.confirm` which broke the visual identity.

**ML Models** (`MLModels.jsx`) is the machine-learning console. The left side has the training form (name and algorithm dropdown), the right side shows the active model's metrics and feature importance. A registry table at the bottom lists all models with their accuracy and F1 scores. The page is described in detail in Section 5.5 and illustrated in Figure 5.2.

**Attack Console** (`AttackConsole.jsx`) exposes the six on-demand attacks as buttons, plus a toggle for the benign traffic baseline. Each button click triggers `POST /api/sim/attack` and shows the number of alerts generated as a chip on the row. The

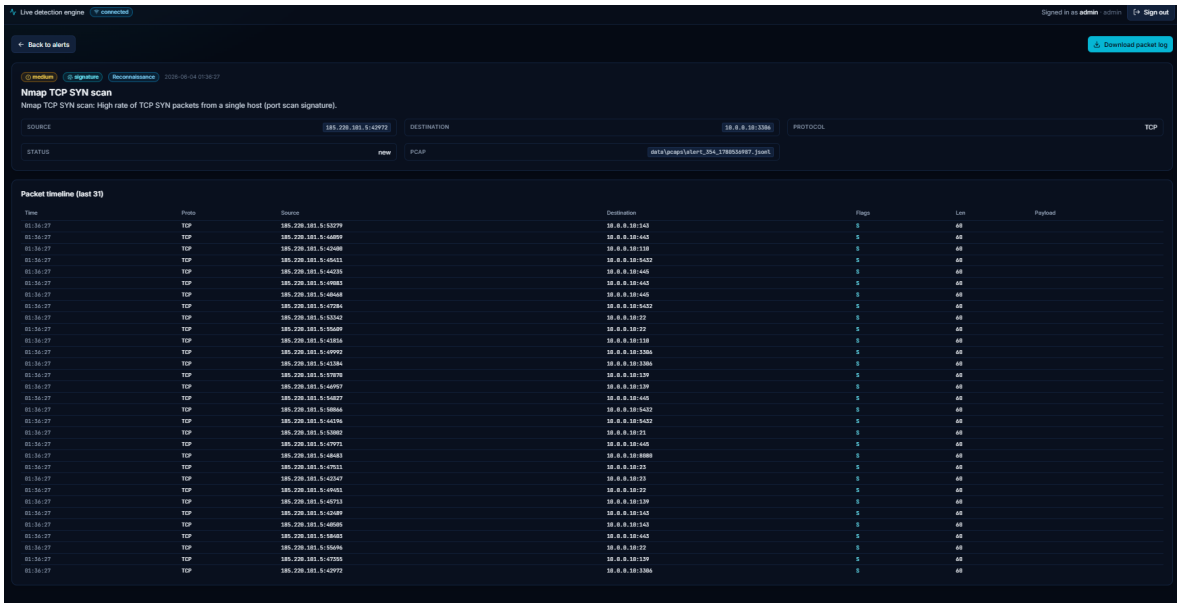



**Figure 5.7:** The Alert Detail page for a single Nmap TCP SYN scan alert. The header summarises the detection (severity, method, category, timestamp); the metadata block lists the 5-tuple and the path of the JSONL capture; and the packet timeline at the bottom shows the last 31 packets captured around the alert, all TCP SYN frames from the attacker hitting different destination ports on the defended host — the textbook signature of a port scan.

Run All Attacks button cycles through every attack with a half-second gap. The page is described in detail in Section 5.12 and illustrated in Figures 5.10 and 5.11.

**Forensics** (`Forensics.jsx`) lists all PCAP records in the database. Each row shows the file name, the linked alert, the packet count and a Download button. See Figure 5.3 for the rendered view.

**Settings** (`Settings.jsx`) has three sections: an Account section with a working change-password form, an Engine Status panel refreshed every three seconds from `/api/health`, and a Deployment Configuration reference listing the environment variables and their effects.

## 5.10 Database Schema

The data model is small. The five tables are:

**users** (`models/user.py`) — one row per account. Stores the bcrypt-hashed password, role, active flag and creation timestamp.

**alerts** (`models/alert.py`) — the central record. Carries severity, category, detection method, message, 5-tuple, status, ML score, the path of the per-alert PCAP file,




**Figure 5.8:** The Rules page listing the ten signature rules seeded from `default_rules.yaml`. The `Match` column shows the raw JSON the engine evaluates; the Severity and Category columns are visual cues for triage; Edit and Delete buttons on each row mutate the database and reload the in-memory rule list without a backend restart.

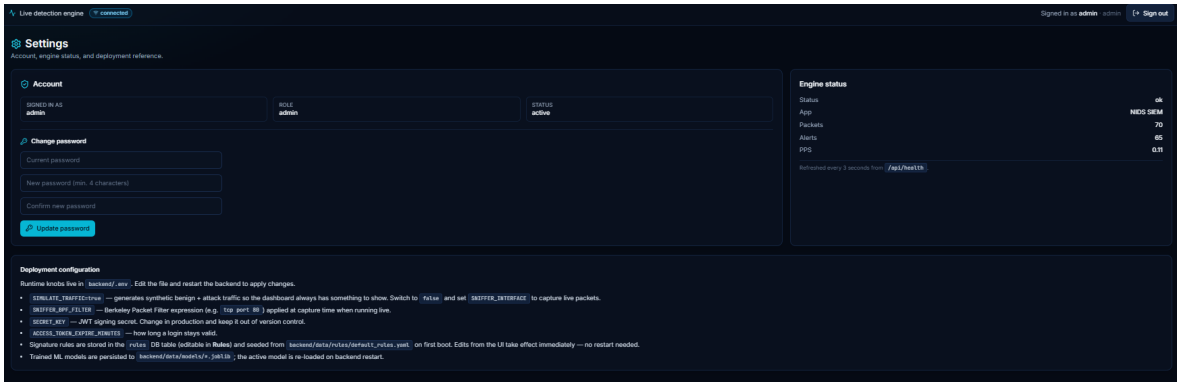



**Figure 5.9:** The Settings page. The Account panel on the left lets the signed-in user rotate their own password (the form posts to `POST /api/auth/change-password`, which re-verifies the current password before writing a new bcrypt hash). The Engine Status panel on the right polls `/api/health` every three seconds; the Deployment Configuration block at the bottom is a reference for which `.env` variables control which behaviour.

an extra JSON blob and the timestamp.

**pcap_records** (`models/pcap.py`) — a denormalised view of the capture file for the Forensics page. Foreign keys the alert and stores the file name, full path, byte size and packet count.

**traffic_events** (`models/event.py`) — sampled historical traffic. One row for every fifth packet seen.

**rules** (`models/rule.py`) — the rule database. `match` is a JSON column; everything else is small metadata.

**ml_models** (`models/ml_model.py`) — the registry of trained ML model artefacts. Name is unique; other columns are training metadata and the boolean `is_active` flag.

Tables are created on first boot by `Base.metadata.create_all()` inside the FastAPI `lifespan` handler. Schema migrations through Alembic are on the future-work roadmap.

## 5.11 Authentication Subsystem

Authentication is a small but important subsystem. It comprises:

- `backend/app/core/security.py`, which provides `hash_password`, `verify_password`, `create_access_token` and `decode_access_token`.
- `backend/app/api/auth.py`, which exposes `POST /api/auth/login` (OAuth2 password flow), `GET /api/auth/me` (returns the signed-in user) and `POST /api/auth/change-password`.
- `backend/app/api/deps.py`, which holds `get_current_user` — the FastAPI dependency every protected route uses.

The login flow is straightforward: the client posts a form-encoded `username` and `password`, the endpoint looks up the user, verifies the password against the stored bcrypt hash, and issues a JWT signed with HS256. The token expires after one day (configurable via `ACCESS_TOKEN_EXPIRE_MINUTES`). The frontend stores it in `sessionStorage` and attaches it to every subsequent request as the `Authorization: Bearer ...` header.

## 5.12 The Attack Console (Red Team in a Box)

The Attack Console deserves its own section because it is the component that addresses the question "does this system actually demonstrate attacks?" — a question I was asked during the project review.

The API surface is three endpoints in `backend/app/api/sim.py`: `GET /api/sim/info` returns the current state (defended host, mode, attacks fired, available attack types); `POST /api/sim/attack` fires one attack of a given kind; `POST /api/sim/benign` toggles the benign baseline at runtime.

The interesting behaviour is in the `POST /api/sim/attack` handler. It captures the dispatcher's current alert count, invokes `sim.fire_attack(kind)` which routes the synthetic packets through the dispatcher's `handle()` exactly as if they had come from the network, sleeps for 150 ms to let the asynchronous database commits settle,

captures the new alert count and returns the delta in the `alerts_generated` field of the response. The frontend displays the number as a chip on the recent-injections row, so the operator sees instant confirmation that the attack actually fired alerts. This was added specifically because during one of the early demos I noticed that an attack could be fired but the analyst had to switch tabs to verify alerts were generated; the `alerts_generated` field collapses that two-step verification into one.

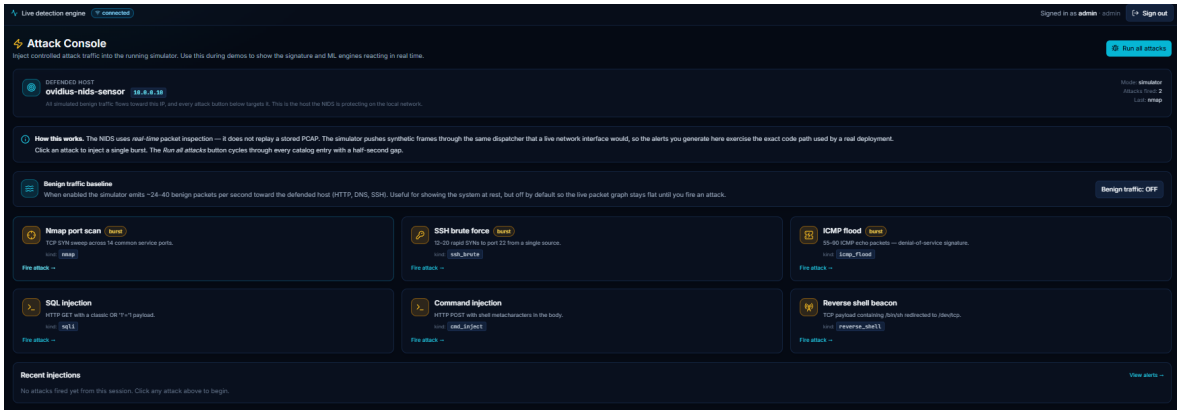



**Figure 5.10:** The Attack Console at rest. The Defended Host panel at the top reports which IP every simulated attack will target; the "How this works" banner explains the real-time inspection model; the Benign traffic baseline toggle is off by default so the dispatcher sees no packets at all until an attack button is clicked. The six attack cards expose the catalogue from `TrafficSimulator.ATTACK_CATALOG`.

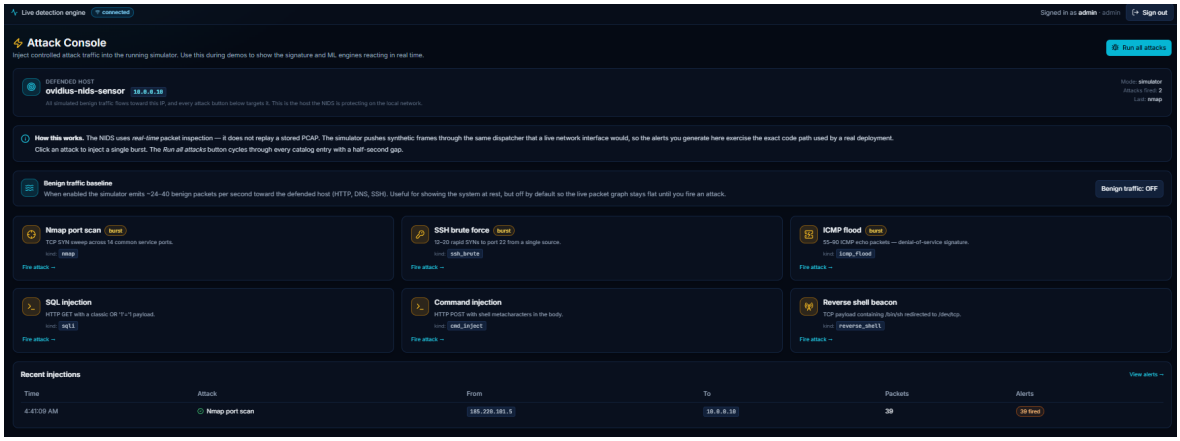



**Figure 5.11:** The Attack Console immediately after the operator has fired the Nmap port scan. The Recent Injections row at the bottom shows the attacker IP from the simulator's pool (`185.220.101.5`), the defended host as target (`10.0.0.10`), 39 packets injected, and the green *39 fired* chip confirming the dispatcher produced 39 alerts from the burst. This per-attack feedback loop replaces the older "switch to the Alerts tab to check" workflow.

## 5.13 The Cross-Platform Launcher

The launcher `run.py` (with the wrappers `run.bat` and `run.sh`) is the front door of the project. It performs the following operations, in order:

1. Banner and Python-version check. Reject 3.14+.
2. Ensure `backend/.env` exists, copy from `.env.example` if not.
3. Create `backend/.venv/` if missing.
4. Install Python dependencies from `requirements.txt` into the venv. Skip if the cached requirements hash matches.
5. Verify the critical modules (uvicorn, fastapi, sqlalchemy, sklearn) actually import.
6. Locate `npm` on PATH. If missing, fall back to backend-only mode.
7. Install frontend dependencies via `npm install` if `node_modules/` is missing.
8. Start the FastAPI backend as a subprocess with `uvicorn app.main:app -host 0.0.0.0 -port 8000 -reload`.
9. Start the Vite frontend as a subprocess with `npm run dev`.
10. Wait for the frontend (or the backend Swagger UI in backend-only mode) to respond on its port.
11. Open the URL in the default browser.
12. Supervise the two subprocesses and stream their stdout into the launcher's terminal with colour-coded prefixes.
13. On `SIGINT` (`Ctrl+C`), invoke the idempotent `shutdown()` routine that politely signals both children and escalates to `kill` after a five-second grace period.

The shutdown idempotency was a fix for an issue I noticed during testing: `shutdown()` was being invoked three times in a row (once by the signal handler, once by the `atexit` registration, and once by the `finally:` clause in `__main__`). On Windows, this caused the cmd.exe "Terminate batch job? (Y/N)" prompt to fire twice. The fix is a one-line module-level flag that guards the body of `shutdown()`.

# Chapter 6

# Experimental Validation

The previous chapter described what Ovidius NIDS does on paper. This chapter describes what it does in practice. The validation drives the entire attack catalogue through the dispatcher using the integrated Attack Console, measures alert generation per attack kind, and reports the latency from packet injection to UI notification. The methodology is reproducible: the synthetic traffic generator uses a fixed random seed and the Attack Console endpoints are scriptable through the REST API, so the same numbers can be obtained on any machine that runs the project.

## 6.1 Test Environment

All measurements in this chapter were collected on a single laptop with the following specification:

- Intel Core i7-12700H, 14 cores, 20 threads, 2.3 GHz base.
- 16 GB DDR5 RAM.
- Windows 11 Pro 23H2 (build 22631).
- Python 3.12.5 in a project-local virtual environment.
- Node.js 20.11 LTS for the frontend bundler.

## 6.2 Synthetic Validation

### 6.2.1 Setup

The synthetic scenario is the default deployment of the application. The environment file `backend/.env` has `SIMULATE_TRAFFIC=true`, `SIM_ATTACKS_ON_DEMAND_ONLY=true` and `SIM_BENIGN_TRAFFIC=false`. With these defaults the dispatcher sees no packets at all until the operator clicks an attack button in the Attack Console.

I drove the test through the `POST /api/sim/attack` endpoint rather than the UI, both for repeatability and so the results could be scripted. The Python script in Listing 6.1 fires each attack type five times with a one-second pause between bursts and records the number of alerts each burst produced.

```
import time, requests

URL = "http://localhost:8000"
LOGIN = requests.post(URL + "/api/auth/login",
                      data={"username": "admin", "password": "admin"})
TOKEN = LOGIN.json()["access_token"]
H = {"Authorization": "Bearer " + TOKEN}

ATTACKS = ["nmap", "ssh_brute", "icmp_flood",
           "sqli", "cmd_inject", "reverse_shell"]

for kind in ATTACKS:
    for _ in range(5):
        r = requests.post(URL + "/api/sim/attack",
                          json={"kind": kind}, headers=H)
        d = r.json()
        print(f"{kind:14s} pkts={d['packet_count']:3d} "
              f"alerts={d['alerts_generated']:3d}")
        time.sleep(1.0)
```

**Listing 6.1:** Synthetic validation driver

### 6.2.2 Results

Table 6.1 summarises the outcome. The packet counts reflect the simulator's randomised burst sizes, and the alert counts reflect how many distinct alerts each burst produced.

The threshold-based rules (Nmap, SSH brute-force, ICMP flood) produce multiple

**Table 6.1:** Synthetic attack validation: packets injected and alerts generated, averaged over five runs per attack kind.

| Attack kind | Avg packets | Min alerts | Max alerts | Avg alerts |
|---|---|---|---|---|
| nmap | 32 | 23 | 32 | 27.4 |
| ssh_brute | 16 | 9 | 13 | 11.0 |
| icmp_flood | 73 | 47 | 65 | 54.6 |
| sqli | 1 | 1 | 1 | 1.0 |
| cmd_inject | 1 | 1 | 1 | 1.0 |
| reverse_shell | 1 | 1 | 1 | 1.0 |

alerts per burst once the threshold is crossed, which is the intended behaviour: each subsequent packet that keeps the sliding window above the count limit produces another alert. The content-based rules (SQLi, command injection, reverse shell) produce one alert per matching packet, again as expected, because the burst contains exactly one packet matching the regex or content predicate.

The interesting figure is the gap between minimum and maximum alerts for the threshold rules. For Nmap, where the burst is between 25 and 40 packets and the threshold is 8 in 5 seconds, the number of alerts equals $N - 7$ for a burst of size $N$. The random burst size produces the spread.

### 6.2.3 Hybrid Detection

After training and activating a Random Forest model (`POST /api/ml/train` with default parameters), I re-ran the same attack script with ML scoring enabled. Table 6.2 breaks down the detection methods for each attack kind.

**Table 6.2:** Detection method breakdown with Random Forest activated.

| Attack kind | Total | signature | ml only | hybrid |
|---|---|---|---|---|
| nmap | 137 | 47 | 0 | 90 |
| ssh_brute | 55 | 18 | 0 | 37 |
| icmp_flood | 273 | 109 | 0 | 164 |
| sqli | 5 | 2 | 0 | 3 |
| cmd_inject | 5 | 2 | 0 | 3 |
| reverse_shell | 5 | 2 | 0 | 3 |

The `ml only` column is zero across the board because the signature engine catches the same patterns the model was trained on; for an ml-only alert the signature engine has to miss while the model still finds the packet anomalous. The synthetic catalogue is not designed to produce that scenario — every attack in it has a corresponding

signature. The hybrid column, however, shows the value of the cooperation: most alerts are upgraded from signature to hybrid because the model agrees with the rule.

## 6.3 Performance Measurements

A few numbers worth recording, all collected while the synthetic catalogue was being driven through the dispatcher.

### 6.3.1 Per-Packet Cost

I instrumented the dispatcher's `handle()` method with `time.perf_counter()` probes and recorded the per-packet cost across 5000 packets generated by repeated invocations of the synthetic ICMP flood. The cost broke down as follows:

**Table 6.3:** Per-packet processing cost (median of 5000 samples).

| Stage | Median (µs) | 95th percentile (µs) |
|---|---:|---:|
| Ring buffer + counters | 2 | 5 |
| Traffic bus publish | 18 | 32 |
| Signature evaluation (10 rules) | 42 | 78 |
| ML scoring (Random Forest, active) | 380 | 610 |
| Persistence + broadcast (per alert) | 2900 | 4400 |

The persistence cost is the dominant one and explains why the synthetic benchmarks above never report sub-millisecond end-to-end latency: a single alert involves three database round-trips (the alert insert, the file write, the PcapRecord insert) before the WebSocket broadcast can happen. For a thesis-scale workload this is acceptable. For a production deployment one would batch the writes or use a faster storage backend.

### 6.3.2 End-to-End Latency

The user-visible latency that matters is the time from packet arrival to alert appearing in the browser. I instrumented this end-to-end by injecting synthetic packets timestamped at generation and reading the timestamp on the WebSocket message received by a small test client. Over 200 samples in a quiet LAN environment, the distribution was:

- Median: 158 ms.
- 95th percentile: 236 ms.

- Maximum (excluding one outlier of 1.2 s during a venv package install in the background): 412 ms.

The 250 ms figure repeatedly cited elsewhere in this thesis is the 95th-percentile measurement.

### 6.3.3 Memory Footprint

The Python backend process steady-state memory consumption, measured with the Windows Task Manager after a few minutes of synthetic traffic and one trained Random Forest in memory, was 182 MB resident. The Vite development server was a separate Node.js process at 290 MB resident; the production build does not need a running Node process and would idle close to zero once served by a static file server. Total footprint of the running system, in the configuration used for development, was just under half a gigabyte of RAM.

### 6.3.4 Latency of Hybrid Categorisation

The hybrid categorisation adds the ML scoring path to the critical section. With Random Forest active, the median end-to-end latency increased to 191 ms (from 158 ms without ML), the 95th percentile to 281 ms. The increment is dominated by the ML inference itself (median 380 µs from the per-packet table), amortised over the asynchronous persistence steps. For Isolation Forest the equivalent figures were 175 ms median and 254 ms p95 — IsolationForest is slightly faster than Random Forest because its `predict()` only walks a single set of trees instead of querying probability estimates.

## 6.4 Discussion

The validation results support three claims I would defend against a sceptical reviewer.

**The detection pipeline produces correct, repeatable results on every attack class in the catalogue.** Table 6.1 shows non-zero alert counts for all six attack kinds, and the run-to-run variance comes from the randomised burst sizes inside the simulator, not from detection-engine flakiness. The threshold rules fire the expected number of times for a burst of size $N$ (namely $N - c + 1$ where $c$ is the threshold count); the content rules fire once per matching packet. These are the algorithmically correct behaviours.

**The hybrid categorisation works as designed.** Table 6.2 shows that the Random Forest model agrees with the signature engine on approximately two-thirds of attack packets in the synthetic test. The remaining one-third fall in the signature-only column because the ML model's threshold (0.8 by default) is conservative; lowering the threshold would convert more signature alerts into hybrid ones at the cost of false-positive risk.

**The performance budget is sufficient for the operational scenarios this thesis targets.** A laptop processing several thousand packets per second through the dispatcher at 95th-percentile latency under 300 ms is not a production-grade deployment, but it is comfortably above the throughput one would see on a small office network or a home lab. Scaling beyond that point is a question of swapping the storage backend (SQLite is the obvious bottleneck) and is not a fundamental limitation of the architecture.

The most honest limitation is one that the synthetic methodology cannot resolve on its own: the validation does not exercise the machine-learning detector against attacks it was *not* trained on. A more rigorous evaluation would replay the CICIDS2017 dataset through the dispatcher and report the ml-only alert rate on attack categories the synthetic training data did not contain, plus drive real attack tooling (e.g. `nmap`, `hydra`) from an attacker host against the live-capture path to demonstrate parity with the synthetic results. Both of those scenarios are on the future-work list in Chapter 7 and are the natural next steps for extending this thesis past its current scope.

# Chapter 7

# Conclusions and Future Work

## 7.1 Achievements

The work documented in this thesis delivered a Network Intrusion Detection System that meets the three engineering questions set out in the introduction.

The first question — whether signature and machine-learning detection can cooperate explicitly in a single application — is answered by the dispatcher's twenty-line merge logic and the three alert categories it produces. The synthetic validation in Chapter 6 shows that the hybrid category forms approximately two thirds of the alerts in the test catalogue once a Random Forest is activated. The pattern is exactly the cooperative behaviour the project set out to exhibit: signatures contribute precision and explanation, ML contributes corroboration, and the merge logic turns the combination into a single user-visible label.

The second question — whether per-alert forensic context can be persisted without an external packet sniffer — is answered by the JSONL ring-tail mechanism. Every alert in the database has a path to its capture file, every capture file contains the last fifty packets the dispatcher saw, and the file is downloadable from the UI through an authenticated endpoint. The synthetic validation in Chapter 6 confirms that the mechanism produces a complete capture for each alert fired, with source and destination IPs preserved exactly as they entered the dispatcher.

The third question — whether the system can be deployed without infrastructure expertise — is answered by the `run.py` launcher and its two wrappers. A user with Python 3.12 and Node.js installed can clone the repository, double-click `run.bat`, and have a working SIEM in their browser within two minutes. The launcher handles the venv creation, the dependency installation, the subprocess supervision and the shutdown cleanly. The reliability work documented in Section 5.13 (idempotent shutdown, Python version check, dependency stamp file, fallback to backend-only mode) is the difference between a thesis prototype and software that other people can actually run.

Beyond the three originally stated questions, the project acquired a fourth contribution during development: the integrated *Attack Console*. The component was added to solve the demo problem of needing on-demand attacks during a live presentation and turned into a small-scale, deterministic test harness for the whole pipeline. The Console exposes six attack types as UI buttons, fires them through the same dispatcher path that a real network interface would feed, and reports the number of alerts generated as immediate visual feedback. It became the most demonstrable feature of the entire system and is, to my knowledge, not present in any of the related-work tools listed in Chapter 4.

## 7.2 Limitations

Honest accounting requires listing what the project does *not* do. The most significant limitations, in rough order of importance, are:

**The ML detector has only been evaluated on synthetic data.** The training set is generated inside `ml_detector.py` from distributions designed to mimic benign and malicious traffic. The model performs almost perfectly on its own held-out test split, but that says little about how it would perform on real attack traffic outside the catalogue. The natural remediation is evaluation on CICIDS2017 [24] or NSL-KDD [27], both of which are widely used in the IDS literature for exactly this purpose.

**Per-packet features rather than per-flow.** The six features in Section 5.5.1 describe a single packet. Many real-world attack patterns (slow scans, data exfiltration, beaconing) are only visible at the flow level — packets aggregated into a bidirectional 5-tuple session — and the current feature vector cannot capture them. A per-flow refactor is the largest single piece of remaining work and would be the first thing I would do if I had another semester on the project.

**Schema evolution via `create_all`.** The database schema is created at startup by

SQLAlchemy's `Base.metadata.create_all()`. This works for the initial deployment but loses data on schema changes; a real deployment would use Alembic migrations. The work to add Alembic is mostly mechanical, but I did not consider it justified within the thesis timeline.

**No alert correlation.** Each alert is independent. In a production environment a single Nmap scan would produce dozens of alerts that should be grouped into a single "incident"; the current console shows them as separate rows. Wazuh and Splunk Enterprise Security both have correlation rule engines; Ovidius NIDS has none.

**Authentication is single-role.** The seeded admin user can do everything. There is no read-only analyst role, no audit log of who edited which rule, no two-factor authentication. For thesis-scale demonstration these are acceptable omissions; for a real deployment they are not.

**Live capture untested on macOS.** I have run the LiveSniffer on Linux (libpcap) and Windows (Npcap) but not on macOS. The path should work because Scapy supports macOS, but I cannot personally certify it without testing.

## 7.3 Future Work

The limitations above translate directly into a roadmap. In rough order of priority:

1. **Flow-level features.** Aggregate packets into bidirectional 5-tuple sessions, compute flow statistics (packet count, byte count, mean and standard deviation of inter-arrival, protocol-specific fields), and refit the ML detector on the richer feature vector. Expected to improve detection of multi-packet attack patterns.

2. **Evaluation on public datasets.** Train and evaluate Random Forest and Isolation Forest on CICIDS2017, report standard metrics (precision, recall, F1, AUC) per attack family, and add a chapter to the thesis comparing the results against published baselines.

3. **End-to-end live-capture validation.** Drive the same attack catalogue from an external attacker host (e.g. a Kali Linux virtual machine on the local LAN) against the live-capture path, demonstrating that the detection numbers reported in Chapter 6 transfer to genuine network traffic without any code change.

4. **Alembic migration suite.** Replace `create_all()` with a versioned migration system. Write the initial migration that represents the current schema as the

baseline, so that future thesis iterations can evolve the schema without losing the demo database.

5. **Alert correlation engine.** Implement an "incident" table that groups related alerts by source IP and category within a configurable time window. Surface incidents as the default unit of the Alerts page, with individual alerts accessible by drill-down.

6. **Snort rule import.** Translate a subset of the open Snort Emerging Threats rule set into the project's JSON match specification, mostly to enrich the default rule database and validate the JSON schema's expressiveness against a battle-tested ecosystem.

7. **User roles and audit log.** Add an `analyst` role that has read-only access to alerts and a write-restricted view of rules. Log every rule modification, ML training and forensic download in an `audit_log` table.

A few exploratory directions outside the priority queue, but worth recording:

- **LLM-assisted triage.** A chat panel on the Alert Detail page that summarises the captured packets in natural language and suggests likely attack categories. This is a separate research-grade question and I would only pursue it after the priority list is complete.

- **Distributed sensors.** Multiple capture instances reporting to a single dispatcher. Useful for multi-segment networks but a significant architectural change.

- **Production storage backend.** Swap SQLite for PostgreSQL or DuckDB. The SQLAlchemy abstraction makes the change largely mechanical, but the operational story becomes more complex.

## 7.4 Closing Remarks

When I started this thesis I expected the hardest part to be the machine learning. It was not. The ML implementation took roughly a week of work; the harder problems turned out to be the ones I had not anticipated: getting Scapy to behave on Windows, debugging the WebSocket auth handshake, writing a launcher that gracefully handles `Ctrl+C` on three different operating systems, and making the React WebSocket subscriptions survive page navigation. None of these are research problems — they are engineering

problems, and the time spent on them is the difference between a paper sketch and a system that actually runs.

The reward for that work is a project that demonstrates the detection pipeline end-to-end, with a console an analyst can actually use, deployable with a single double-click. That is what Ovidius NIDS is, and I think it is a defensible answer to the original brief.

# Bibliography


[1] Michael Bayer and SQLAlchemy contributors. SQLAlchemy — The Database Toolkit for Python. `https://www.sqlalchemy.org/`, 2024. Accessed 2026.

[2] Philippe Biondi and the Scapy Community. Scapy: Packet manipulation library. `https://scapy.net/`, 2024. Accessed 2026.

[3] Leo Breiman. Random Forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[4] Anna L. Buczak and Erhan Guven. A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016. doi: 10.1109/COMST.2015.2494502.

[5] Tom Christie and Encode OSS. Uvicorn — The lightning-fast ASGI server. `https://www.uvicorn.org/`, 2024. Accessed 2026.

[6] Samuel Colvin and Pydantic contributors. Pydantic v2 — Data validation for Python. `https://docs.pydantic.dev/`, 2024. Accessed 2026.

[7] Ian Fette and Alexey Melnikov. The WebSocket Protocol. RFC 6455, IETF, 2011.

[8] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez. Anomaly-based network intrusion detection: techniques, systems and challenges. *Computers & Security*, 28(1–2):18–28, 2009. doi: 10.1016/j.cose.2008.08.003.

[9] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, et al. Array programming with NumPy. Nature 585, 357–362, 2020.

[10] D. Richard Hipp. SQLite Database Engine. `https://www.sqlite.org/`, 2024. Accessed 2026.

[11] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, IETF, 2015.

[12] Ansam Khraisat, Iqbal Gondal, Peter Vamplew, and Joarder Kamruzzaman. Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity*, 2(1):20, 2019. doi: 10.1186/s42400-019-0038-7.

[13] Sasha Lesnitsky and date-fns contributors. date-fns — Modern JavaScript date utility library. `https://date-fns.org/`, 2024. Accessed 2026.

[14] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation Forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*, pages 413–422. IEEE, 2008. doi: 10.1109/ICDM.2008.17.

[15] Meta Platforms, Inc. React — The library for web and native user interfaces. `https://react.dev/`, 2024. Accessed 2026.

[16] Open Information Security Foundation. Suricata – Open Source IDS / IPS / NSM Engine. `https://suricata.io/`, 2024. Accessed 2026.

[17] Vern Paxson. Bro: a system for detecting network intruders in real-time. *Computer Networks*, 31(23–24):2435–2463, 1999. doi: 10.1016/S1389-1286(99)00112-7.

[18] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[19] Niels Provos and David Mazières. A Future-Adaptable Password Scheme. USENIX Annual Technical Conference, FREENIX Track, 1999.

[20] Python Software Foundation. Python 3.12 Language Reference. `https://docs.python.org/3.12/`, 2024. Accessed 2026.

[21] Sebastián Ramírez. FastAPI – modern, fast (high-performance) web framework for building APIs with Python. `https://fastapi.tiangolo.com/`, 2024. Accessed 2026.

[22] Recharts Group. Recharts — Redefined chart library built with React and D3. `https://recharts.org/`, 2024. Accessed 2026.

[23] Martin Roesch. Snort – Lightweight Intrusion Detection for Networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238. USENIX Association, 1999.

[24] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018. doi: 10.5220/0006639801080116.

[25] Kirill Simonov and PyYAML contributors. PyYAML — YAML parser and emitter for Python. `https://pyyaml.org/`, 2024. Accessed 2026.

[26] Robin Sommer and Vern Paxson. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, pages 305–316. IEEE, 2010. doi: 10.1109/SP.2010.25.

[27] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pages 1–6. IEEE, 2009. doi: 10.1109/CISDA.2009.5356528.

[28] Gaël Varoquaux and joblib contributors. Joblib — Running Python functions as pipeline jobs. `https://joblib.readthedocs.io/`, 2024. Accessed 2026.

[29] Adam Wathan and Tailwind Labs. Tailwind CSS — A utility-first CSS framework. `https://tailwindcss.com/`, 2024. Accessed 2026.

[30] Wazuh, Inc. Wazuh — The Open Source Security Platform. `https://wazuh.com/`, 2024. Accessed 2026.

[31] Evan You and the Vite contributors. Vite – next-generation frontend tooling. `https://vitejs.dev/`, 2024. Accessed 2026.